

N_Body_Opt

May 25, 2026

```
[ ]: # Import everything I might need, I normally just copy and paste this from ↵
      ↪previous notebooks just in case lol
import matplotlib.pyplot as plt
import matplotlib.transforms as transforms
%matplotlib inline
from scipy.integrate import solve_ivp
from scipy.optimize import root_scalar, curve_fit
import numpy as np
import csv
import math
import scipy.integrate as integrate
import scipy.special as special
from scipy.signal import find_peaks
import astropy.constants as c
import astropy.units as u
import astropy.io.fits as pyfits
from astropy.io import fits
from scipy.interpolate import RectBivariateSpline
from scipy import interpolate
from scipy.special import gamma, gammainc, gammaincc
from matplotlib import ticker
from matplotlib.patches import Rectangle
import pandas as pd
import csv
import seaborn as sns
from matplotlib.lines import Line2D
from scipy.stats import ttest_ind
from scipy.stats import spearmanr, pearsonr

# The below code in this cell was written with the assistance of Gen-AI to help ↵
      ↪visualise my results as suggested by tutor
# Visual style helpers

GALAXY_1_COLOR = "#ff5a5f"
GALAXY_2_COLOR = "#4dabf7"
DARK_BACKGROUND = "#05070d"
DARK_PANEL = "#0b1020"
```

```

DARK_GRID = "#26324a"
DARK_TEXT = "#e8eefc"

plt.rcParams.update(
    {
        "figure.facecolor": DARK_BACKGROUND,
        "axes.facecolor": DARK_PANEL,
        "axes.edgecolor": DARK_TEXT,
        "axes.labelcolor": DARK_TEXT,
        "xtick.color": DARK_TEXT,
        "ytick.color": DARK_TEXT,
        "text.color": DARK_TEXT,
        "axes.titlecolor": DARK_TEXT,
        "legend.facecolor": DARK_PANEL,
        "legend.edgecolor": DARK_GRID,
        "legend.labelcolor": DARK_TEXT,
        "grid.color": DARK_GRID,
        "savefig.facecolor": DARK_BACKGROUND,
    }
)

def galaxy_color_array(n_particles):
    """
    Return red/blue colours for particles from the two galaxies.

    The initialization functions concatenate galaxy 1 followed by galaxy 2, so
    the first half of the particle array is coloured red and the second half is
    coloured blue.
    """

    n_first = n_particles // 2
    n_second = n_particles - n_first
    return [GALAXY_1_COLOR] * n_first + [GALAXY_2_COLOR] * n_second

def style_dark_axis(ax, grid=True):
    """
    Apply the dark plotting style to one Matplotlib axis.
    """

    ax.set_facecolor(DARK_PANEL)
    ax.tick_params(colors=DARK_TEXT)
    ax.xaxis.label.set_color(DARK_TEXT)
    ax.yaxis.label.set_color(DARK_TEXT)
    ax.title.set_color(DARK_TEXT)

```

```

for spine in ax.spines.values():
    spine.set_color(DARK_TEXT)

if grid:
    ax.grid(True, color=DARK_GRID, alpha=0.7, linewidth=0.8)

def scatter_galaxy_positions(ax, positions, s=5, alpha=0.9):
    """
    Plot particle positions using separate colours for the two galaxies.
    """

    positions = np.asarray(positions)
    colours = galaxy_color_array(len(positions))

    return ax.scatter(
        positions[:, 0],
        positions[:, 1],
        s=s,
        c=colours,
        alpha=alpha,
        edgecolors="none"
    )

def scatter_particle_list(ax, particles, s=5, alpha=0.9):
    """
    Plot a list of Particle objects using the two-galaxy colour scheme.
    """

    positions = np.array([p.position for p in particles])
    return scatter_galaxy_positions(ax, positions, s=s, alpha=alpha)

```

```

[1]: # Skeleton Code given copied and then edited and adapted
class QuadTreeNode:
    """
    A safer quadtree node for the Barnes-Hut algorithm.

    This version prevents infinite subdivision when two particles have
    identical or nearly identical positions.
    """

    def __init__(self, x_min, x_max, y_min, y_max, depth=0, max_depth=40,
    ↪min_size=1e-10):
        """
        Initialize the boundary box and node properties.

```

```

    depth:
        Current depth of this node in the tree.

    max_depth:
        Maximum allowed subdivision depth. This prevents infinite recursion.

    min_size:
        Minimum allowed node size. If the box becomes smaller than this,
        we stop subdividing.
    """

    self.bbox = np.array([x_min, x_max, y_min, y_max], dtype=float)

    self.children = None
    self.total_mass = None
    self.center_of_mass = np.zeros(2)

    self.particle_position = None
    self.particle_mass = None

    self.depth = depth
    self.max_depth = max_depth
    self.min_size = min_size

def update_center_of_mass(self, position, mass):
    """
    Update the total mass and center of mass after adding a particle.
    """

    position = np.array(position, dtype=float)

    if self.total_mass is None:
        self.total_mass = mass
        self.center_of_mass = position.copy()

    else:
        old_mass = self.total_mass
        new_mass = old_mass + mass

        self.center_of_mass = (
            old_mass * self.center_of_mass + mass * position
        ) / new_mass

        self.total_mass = new_mass

```

```

def get_quadrant(self, position):
    """
    Return which quadrant contains the given position.

    Quadrant convention:
        0 = bottom-left
        1 = bottom-right
        2 = top-left
        3 = top-right
    """

    x_min, x_max, y_min, y_max = self.bbox
    x_mid = 0.5 * (x_min + x_max)
    y_mid = 0.5 * (y_min + y_max)

    x, y = position

    if x < x_mid and y < y_mid:
        return 0
    elif x >= x_mid and y < y_mid:
        return 1
    elif x < x_mid and y >= y_mid:
        return 2
    else:
        return 3

def subdivide(self, quadrant):
    """
    Create a child node in the selected quadrant.
    """

    x_min, x_max, y_min, y_max = self.bbox
    x_mid = 0.5 * (x_min + x_max)
    y_mid = 0.5 * (y_min + y_max)

    if quadrant == 0:
        child_bbox = (x_min, x_mid, y_min, y_mid)

    elif quadrant == 1:
        child_bbox = (x_mid, x_max, y_min, y_mid)

    elif quadrant == 2:
        child_bbox = (x_min, x_mid, y_mid, y_max)

    elif quadrant == 3:
        child_bbox = (x_mid, x_max, y_mid, y_max)

```

```

else:
    raise ValueError("Quadrant must be 0, 1, 2, or 3.")

self.children[quadrant] = QuadTreeNode(
    *child_bbox,
    depth=self.depth + 1,
    max_depth=self.max_depth,
    min_size=self.min_size
)

def insert(self, position, mass):
    """
    Insert a particle into the quadtree.

    This version includes a safety condition, a method which was suggested_
    ↪by Gen-AI:
    if two particles are too close together, or the tree becomes too deep,
    we do not keep subdividing forever. Instead, we combine their mass
    into the same leaf node.
    """

    position = np.array(position, dtype=float)

    x_min, x_max, y_min, y_max = self.bbox
    node_size = max(x_max - x_min, y_max - y_min)

    # Case 1: empty node
    if self.total_mass is None:
        self.update_center_of_mass(position, mass)
        self.particle_position = position.copy()
        self.particle_mass = mass
        return

    # Safety case:
    # If the node is already extremely small, or if we reached max_depth,
    # stop subdividing and simply merge the new particle into this node
    if self.depth >= self.max_depth or node_size < self.min_size:
        self.update_center_of_mass(position, mass)

        # Since this leaf now represents more than one unresolved particle,
        # we remove the single-particle marker
        self.particle_position = None
        self.particle_mass = None
        return

```

```

# Case 2: occupied leaf node
if self.children is None:

    old_position = self.particle_position
    old_mass = self.particle_mass

    # Extra safety:
    # If the old and new positions are essentially identical,
    # merge them instead of subdividing forever
    if old_position is not None and np.linalg.norm(position -
↪old_position) < self.min_size:
        self.update_center_of_mass(position, mass)
        self.particle_position = None
        self.particle_mass = None
        return

    self.children = [None, None, None, None]

    self.particle_position = None
    self.particle_mass = None

    # Insert the old particle into its child
    old_quadrant = self.get_quadrant(old_position)
    if self.children[old_quadrant] is None:
        self.subdivide(old_quadrant)
    self.children[old_quadrant].insert(old_position, old_mass)

    # Insert the new particle into its child
    new_quadrant = self.get_quadrant(position)
    if self.children[new_quadrant] is None:
        self.subdivide(new_quadrant)
    self.children[new_quadrant].insert(position, mass)

    # Update this parent node with the new particle
    self.update_center_of_mass(position, mass)
    return

# Case 3: already subdivided node
quadrant = self.get_quadrant(position)

if self.children[quadrant] is None:
    self.subdivide(quadrant)

self.children[quadrant].insert(position, mass)

self.update_center_of_mass(position, mass)

```

0.0.1 Breakdown of code cell above

This cell is organized into the following pieces:

- Defines `QuadTreeNode`, which stores part of the simulation state or Barnes-Hut data structure.
- Defines reusable functions used by later cells:
 - `QuadTreeNode.__init__` supports the simulation workflow.
 - `QuadTreeNode.update_center_of_mass` updates a node's total mass and centre of mass when a particle is inserted.
 - `QuadTreeNode.get_quadrant` decides which quadrant of a node contains a particle.
 - `QuadTreeNode.subdivide` creates child nodes for quadtree subdivision.
 - `QuadTreeNode.insert` places particles into the quadtree while handling occupied leaves.

```
[2]: # Now to initialise this we need to create our final classes and start with the
      ↪ most simple cases
class Particle:
    """
    Simple particle class for the N-body simulation.

    Each particle has:
        position : 2D NumPy array [x, y]
        velocity : 2D NumPy array [vx, vy]
        mass      : scalar mass
    """

    def __init__(self, position, velocity, mass):
        self.position = np.array(position, dtype=float)
        self.velocity = np.array(velocity, dtype=float)
        self.mass = float(mass)

class QuadTree:
    """
    A class that represents the full tree, which contains all the nodes
    and particles.
    """

    def __init__(self, particles, theta):
        """
        Initialize the full Barnes-Hut tree.

        particles : list
            List of Particle objects.
        theta : float
            Barnes-Hut opening angle / accuracy parameter.
        """

        self.particles = particles
```



```

self.theta = theta

# Extract all particle positions into arrays
positions = np.array([p.position for p in particles])

# Find spatial limits of all particles
x_min, y_min = np.min(positions, axis=0)
x_max, y_max = np.max(positions, axis=0)

# Add padding so particles are not exactly on the boundary
padding = 0.1 * max(x_max - x_min, y_max - y_min)

if padding == 0:
    padding = 1.0

x_min -= padding
x_max += padding
y_min -= padding
y_max += padding

# Barnes-Hut works most cleanly with square boxes
# So we enlarge the shorter side to make the root node square
x_center = 0.5 * (x_min + x_max)
y_center = 0.5 * (y_min + y_max)

box_size = max(x_max - x_min, y_max - y_min)

x_min = x_center - 0.5 * box_size
x_max = x_center + 0.5 * box_size
y_min = y_center - 0.5 * box_size
y_max = y_center + 0.5 * box_size

# Create the root node covering the whole simulation region
self.root = QuadTreeNode(x_min, x_max, y_min, y_max)

# Insert all particles into the tree
self.insert()

def insert(self):
    """
    Insert particles one by one into the root node of the tree.
    """

    for particle in self.particles:
        self.root.insert(particle.position, particle.mass)

```

```

def get_quadrant(bbox, position):
    """
    Determine which quadrant of a bounding box contains a given position.

    Quadrant convention:
        0 = bottom-left
        1 = bottom-right
        2 = top-left
        3 = top-right
    """

    x_min, x_max, y_min, y_max = bbox

    x_mid = 0.5 * (x_min + x_max)
    y_mid = 0.5 * (y_min + y_max)

    x, y = position

    if x < x_mid and y < y_mid:
        return 0

    elif x >= x_mid and y < y_mid:
        return 1

    elif x < x_mid and y >= y_mid:
        return 2

    else:
        return 3


def initialize_particles(N):
    """
    Initialize  $N$  particles using a Plummer-like model in  $N$ -body units.

    Assumptions:
         $G = 1$ 
        total mass  $M = 1$ 
        virial radius  $r_v = 1$ 

    Each particle has equal mass:
         $m = 1 / N$ 
    """

    particles = []

```

```

# N-body units
M = 1.0
r_v = 1.0

# we are given this in the our PDF so the basic math is just as follows (I
→should remember to include this in report):
#  $r_v = 16a / (3\pi)$ 
# Therefore:
#  $a = 3\pi r_v / 16$ 
a = 3 * np.pi * r_v / 16

# Equal particle masses
particle_mass = M / N

for i in range(N):

    # Random number for inverse Plummer radius sampling
    # Avoid exactly 0 because it would cause division problems
    p = np.random.uniform(1e-10, 1.0)

    # Standard inverse sampling formula for a Plummer sphere:
    #  $r = a / \sqrt{p^{-2/3} - 1}$ 
    r = a / np.sqrt(p**(-2.0 / 3.0) - 1.0)

    # Random angular variables.
    p0 = np.random.uniform(0.0, 1.0)
    p1 = np.random.uniform(0.0, 1.0)

    # Random direction.
    # This follows the project description, using a random direction
    # and keeping the x-y coordinates
    theta = np.arccos(2.0 * p0 - 1.0)
    phi = 2.0 * np.pi * p1

    x = r * np.sin(theta) * np.cos(phi)
    y = r * np.sin(theta) * np.sin(phi)

    position = np.array([x, y])

    # For now, initialize with zero velocity.
    # Later we can replace this with rotational or differential rotation
    velocity = np.array([0.0, 0.0])

    particle = Particle(position, velocity, particle_mass)
    particles.append(particle)

return particles

```

0.0.2 Breakdown of code cell above

This cell is organized into the following pieces:

- Defines `Particle`, which stores part of the simulation state or Barnes-Hut data structure.
- Defines `QuadTree`, which stores part of the simulation state or Barnes-Hut data structure.
- Defines reusable functions used by later cells:
 - `Particle.__init__` supports the simulation workflow.
 - `QuadTree.__init__` supports the simulation workflow.
 - `QuadTree.insert` places particles into the quadtree while handling occupied leaves.
 - `get_quadrant` decides which quadrant of a node contains a particle.
 - `initialize_particles` samples a Plummer-like particle distribution in N-body units.
- Generates particle positions from a Plummer-like distribution and assigns equal particle masses.

```
[3]: # Now lets actually initialise and create our galaxies given we now have our
      ↪ particles
def initialize_two_galaxies(
    N_per_galaxy=50,
    d=5.0,
    theta=0.5,
    v_offset=0.2
):
    """
    Initialize two Plummer-model galaxies separated by distance d.

    Parameters
    -----
    N_per_galaxy : int
        Number of particles in each galaxy.

    d : float
        Distance between the centers of the two galaxies.

    theta : float
        Barnes-Hut opening angle parameter. This is not used directly to create
        the particles, but is returned so it can be passed into QuadTree.

    v_offset : float
        Initial bulk velocity magnitude given to each galaxy so that they move
        toward each other.

    Returns
    -----
    particles : list
        Combined list of Particle objects from both galaxies.

    theta : float
```

```

Barnes-Hut opening angle parameter.
"""

# Create the first galaxy centered initially around (0, 0)
galaxy_1 = initialize_particles(N_per_galaxy)

# Create the second galaxy centered initially around (0, 0)
galaxy_2 = initialize_particles(N_per_galaxy)

# We want the two galaxy centers to be separated by distance d
# Place galaxy 1 at  $x = -d/2$  and galaxy 2 at  $x = +d/2$ .
offset_1 = np.array([-d / 2.0, 0.0])
offset_2 = np.array([ d / 2.0, 0.0])

# Shift galaxy 1 to the left.
# Give it a velocity to the right so it moves toward galaxy 2
for particle in galaxy_1:
    particle.position += offset_1
    particle.velocity += np.array([v_offset, 0.0])

# Shift galaxy 2 to the right.
# Give it a velocity to the left so it moves toward galaxy 1
for particle in galaxy_2:
    particle.position += offset_2
    particle.velocity += np.array([-v_offset, 0.0])

# Combine the two galaxies into one particle list
particles = galaxy_1 + galaxy_2

return particles, theta

```

0.0.3 Breakdown of code cell above

This cell is organized into the following pieces:

- Defines reusable functions used by later cells:
 - `initialize_two_galaxies` builds two separated Plummer systems with bulk approach velocities.
- Sets up two Plummer-like galaxies separated in space and gives them bulk velocities toward one another.

```

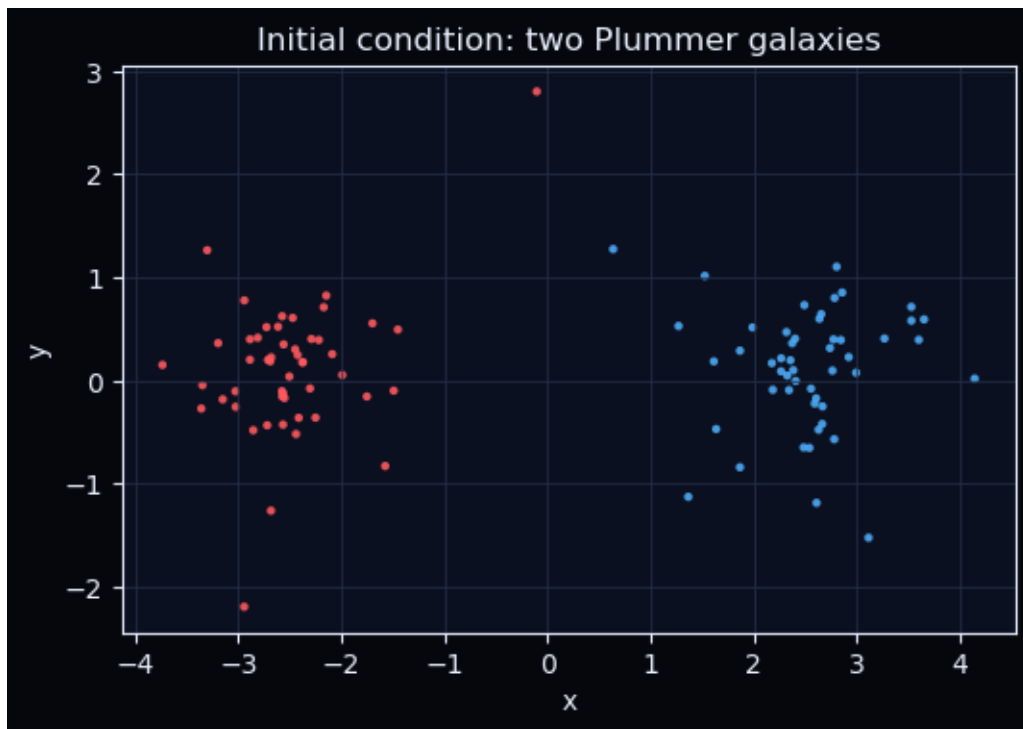
[4]: # Lets run this and see if we actually get any errors or if it runs?
particles, theta = initialize_two_galaxies(
    N_per_galaxy=50,
    d=5.0,
    theta=0.5,
    v_offset=0.2
)

```

```
tree = QuadTree(particles, theta)
```

```
[5]: # Lets plot to see if it actually does what we expect
positions = np.array([p.position for p in particles])

fig, ax = plt.subplots(figsize=(6, 6))
scatter_galaxy_positions(ax, positions, s=10)
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("Initial condition: two Plummer galaxies")
ax.set_aspect("equal")
style_dark_axis(ax)
plt.show()
```



```
[6]: # Different particle numbers to test
# These are particles PER galaxy so the total number of particles is double this
particle_numbers = [50, 100, 250, 500, 1000]

# Create a figure with multiple subplots
# Create or configure the Matplotlib figure
fig, axes = plt.subplots(1, len(particle_numbers), figsize=(20, 4))
fig.patch.set_facecolor(DARK_BACKGROUND)
```

```

# Loop over the different particle counts
for ax, N in zip(axes, particle_numbers):

    # Generate two galaxies with N particles each
    particles, theta = initialize_two_galaxies(
        N_per_galaxy=N,
        d=5.0,
        theta=0.5,
        v_offset=0.2
    )

    # Scatter plot of particle positions, coloured by parent galaxy
    positions = np.array([p.position for p in particles])
    scatter_galaxy_positions(ax, positions, s=2)

    # Title showing particle count
    ax.set_title(f"N = {N}")

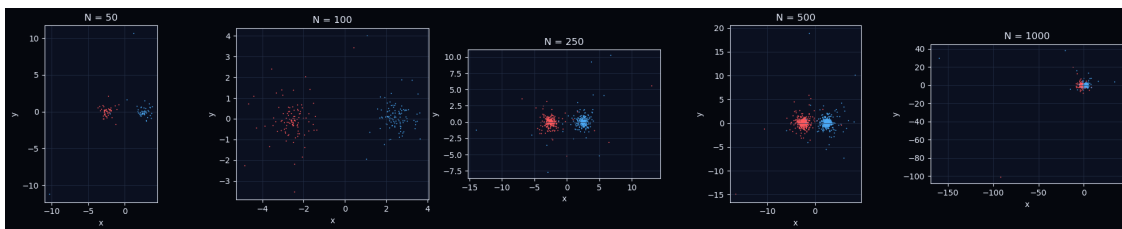
    # Keep aspect ratio square so galaxies are not distorted
    ax.set_aspect("equal")

    # Label axes.
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    style_dark_axis(ax)

plt.tight_layout()

plt.show()

```



[7]: *# This code cell was written with assistance of Gen-AI as ran into an error*
→ that I could not fix when implementing my RK4 code below
Gen-AI suggested this safer approach to fix these cases
As you can see some of these safer functions are directly written by Gen-AI
→ as it is just to resolve a python error

```

def safer_insert(self, position, mass):

```

```

"""
Safer replacement for QuadTreeNode.insert.

This version handles three cases:
1. Empty leaf node
2. Leaf node containing one real particle
3. Leaf node representing multiple merged/unresolved particles

"""

position = np.array(position, dtype=float)

x_min, x_max, y_min, y_max = self.bbox
node_size = max(x_max - x_min, y_max - y_min)

# -----
# Case 1: this node is completely empty
# -----
if self.total_mass is None:
    self.update_center_of_mass(position, mass)
    self.particle_position = position.copy()
    self.particle_mass = mass
    return

# -----
# Safety case: this node is already too small/deep
# -----
if self.depth >= self.max_depth or node_size < self.min_size:
    self.update_center_of_mass(position, mass)

    # This node now represents multiple unresolved particles.
    self.particle_position = None
    self.particle_mass = None
    return

# -----
# Case 2: this is a leaf node
# -----
if self.children is None:

    # IMPORTANT FIX:
    # If particle_position is None, this leaf already represents
    # multiple unresolved particles. There is no single old particle
    # to redistribute, so we simply add the new mass here.
    if self.particle_position is None:
        self.update_center_of_mass(position, mass)

```



```

        return

    old_position = self.particle_position.copy()
    old_mass = self.particle_mass

    # If the new particle is essentially on top of the old one,
    # merge them instead of subdividing forever.
    if np.linalg.norm(position - old_position) < self.min_size:
        self.update_center_of_mass(position, mass)
        self.particle_position = None
        self.particle_mass = None
        return

    # Otherwise subdivide this node.
    self.children = [None, None, None, None]

    # This node is no longer a single-particle leaf.
    self.particle_position = None
    self.particle_mass = None

    # Insert the old particle into the correct child.
    old_quadrant = self.get_quadrant(old_position)
    if self.children[old_quadrant] is None:
        self.subdivide(old_quadrant)
    self.children[old_quadrant].insert(old_position, old_mass)

    # Insert the new particle into the correct child.
    new_quadrant = self.get_quadrant(position)
    if self.children[new_quadrant] is None:
        self.subdivide(new_quadrant)
    self.children[new_quadrant].insert(position, mass)

    # Update this parent node to include the new particle.
    self.update_center_of_mass(position, mass)
    return

# -----
# Case 3: this node is already subdivided
# -----
    quadrant = self.get_quadrant(position)

    if self.children[quadrant] is None:
        self.subdivide(quadrant)

    self.children[quadrant].insert(position, mass)

    self.update_center_of_mass(position, mass)

```

```
# Replace the old insert method with the safer one.
QuadTreeNode.insert = safer_insert
```

[8]: *# Barnes-Hut force calculation + RK4 N-body Integrator*

```
def compute_force_from_node(node, particle, theta=0.5, G=1.0, eps=0.01):
    """
    Compute the Barnes-Hut acceleration on one particle.
    """

    if node is None or node.total_mass is None:
        return np.zeros(2)

    r_vec = node.center_of_mass - particle.position
    r = np.sqrt(np.sum(r_vec**2) + eps**2)

    # Avoid self-force for a normal one-particle leaf node
    if node.children is None and node.particle_position is not None:
        if np.linalg.norm(node.particle_position - particle.position) < 1e-12:
            return np.zeros(2)

    x_min, x_max, y_min, y_max = node.bbox
    L = max(x_max - x_min, y_max - y_min)

    # If this is a leaf node, use its mass directly
    if node.children is None:
        return G * node.total_mass * r_vec / r**3

    # Barnes-Hut approximation
    if L / r < theta:
        return G * node.total_mass * r_vec / r**3

    # Otherwise, open the node and sum child contributions
    acceleration = np.zeros(2)

    for child in node.children:
        if child is not None:
            acceleration += compute_force_from_node(
                child, particle, theta=theta, G=G, eps=eps
            )

    return acceleration
```

```

def compute_accelerations(particles, theta=0.5, G=1.0, eps=0.01):
    """
    Compute accelerations for all particles using the Barnes-Hut tree.

    A new tree is built from the current particle positions, then the
    acceleration on each particle is computed from the tree.

    Returns
    -----
    accelerations : ndarray
        Array of shape (N, 2), containing acceleration vectors.
    """

    # Build the Barnes-Hut tree for the current particle positions
    tree = QuadTree(particles, theta)

    # Compute acceleration on each particle using the root of the tree
    accelerations = []

    for particle in particles:
        acc = compute_force_from_node(
            tree.root, particle, theta=theta, G=G, eps=eps
        )
        accelerations.append(acc)

    return np.array(accelerations)

def get_state(particles):
    """
    Convert the list of Particle objects into position, velocity, and mass
    arrays.

    This is useful to have in arrays for later and an idea suggested by Gen-AI.
    """

    positions = np.array([p.position for p in particles])
    velocities = np.array([p.velocity for p in particles])
    masses = np.array([p.mass for p in particles])

    return positions, velocities, masses

def set_state(particles, positions, velocities):
    """
    Update the Particle objects using new position and velocity arrays.
    """

```

```

for i, particle in enumerate(particles):
    particle.position = positions[i].copy()
    particle.velocity = velocities[i].copy()

def temporary_particles(positions, velocities, masses):
    """
    Create a temporary list of Particle objects from arrays.

    RK4 needs to evaluate accelerations at intermediate positions.
    This function lets us build temporary particle states without permanently
    changing the original simulation particles.
    """

    temp = []

    for i in range(len(masses)):
        temp.append(Particle(positions[i], velocities[i], masses[i]))

    return temp

def derivatives(positions, velocities, masses, theta=0.5, G=1.0, eps=0.01):
    """
    Compute the derivatives needed by RK4.

    For an N-body system:
         $d(\text{position})/dt = \text{velocity}$ 
         $d(\text{velocity})/dt = \text{acceleration}$ 

    Returns
    -----
    dpos_dt : ndarray
        Time derivative of positions.

    dvel_dt : ndarray
        Time derivative of velocities.
    """

    # Make temporary particles at the current RK4 sub-step state
    temp = temporary_particles(positions, velocities, masses)

    # Velocity is the derivative of position
    dpos_dt = velocities

    # Acceleration is the derivative of velocity

```

```

dvel_dt = compute_accelerations(temp, theta=theta, G=G, eps=eps)

return dpos_dt, dvel_dt

def rk4_step(particles, dt, theta=0.5, G=1.0, eps=0.01):
    """
    Advance the whole N-body system forward by one RK4 timestep.

    RK4 evaluates the derivative four times:
        k1 at the start
        k2 at the midpoint
        k3 at the midpoint again
        k4 at the end

    Then it combines them in a weighted average.
    """

    # Convert current particle data into arrays
    pos, vel, masses = get_state(particles)

    # k1: derivative at the beginning of the interval
    k1_pos, k1_vel = derivatives(pos, vel, masses, theta=theta, G=G, eps=eps)

    # k2: derivative at the midpoint using k1
    k2_pos, k2_vel = derivatives(
        pos + 0.5 * dt * k1_pos,
        vel + 0.5 * dt * k1_vel,
        masses,
        theta=theta,
        G=G,
        eps=eps
    )

    # k3: another midpoint estimate using k2
    k3_pos, k3_vel = derivatives(
        pos + 0.5 * dt * k2_pos,
        vel + 0.5 * dt * k2_vel,
        masses,
        theta=theta,
        G=G,
        eps=eps
    )

    # k4: derivative at the end of the interval using k3
    k4_pos, k4_vel = derivatives(
        pos + dt * k3_pos,

```

```

        vel + dt * k3_vel,
        masses,
        theta=theta,
        G=G,
        eps=eps
    )

    # Weighted RK4 update for positions
    new_pos = pos + (dt / 6.0) * (
        k1_pos + 2.0 * k2_pos + 2.0 * k3_pos + k4_pos
    )

    # Weighted RK4 update for velocities
    new_vel = vel + (dt / 6.0) * (
        k1_vel + 2.0 * k2_vel + 2.0 * k3_vel + k4_vel
    )

    # Write the updated arrays back into the Particle objects
    set_state(particles, new_pos, new_vel)

def evolve_fixed_timestep(
    particles,
    t_end,
    dt,
    theta=0.5,
    G=1.0,
    eps=0.01,
    save_every=1
):
    """
    Evolve the system using RK4 with a fixed timestep.

    Parameters
    -----
    particles : list
        List of Particle objects.

    t_end : float
        Final simulation time.

    dt : float
        Fixed timestep size.

    theta : float
        Barnes-Hut opening parameter.

```

```

    save_every : int
        Save every save_every steps.

    Returns
    -----
    times : ndarray
        Saved times.

    history : ndarray
        Saved particle positions with shape:
        (number of saved frames, number of particles, 2)
    """

    times = []
    history = []

    t = 0.0
    step = 0

    while t < t_end:

        # Save positions for plotting or animation
        if step % save_every == 0:
            times.append(t)
            history.append(np.array([p.position.copy() for p in particles]))

        # Make sure the final step lands exactly on t_end
        current_dt = min(dt, t_end - t)

        # Advance one RK4 step
        rk4_step(particles, current_dt, theta=theta, G=G, eps=eps)

        t += current_dt
        step += 1

    return np.array(times), np.array(history)

# Instead of fixed dt lets use an adaptive one to investigate if it is different
def choose_adaptive_dt(
    particles,
    dt_max=0.02,
    dt_min=0.002,
    eta=0.2,
    theta=0.5,
    G=1.0,
    eps=0.05

```

```

):
    """
    Choose an adaptive timestep based on the strongest acceleration.

    Larger acceleration means smaller timestep.
    This version uses less aggressive settings so it runs faster.
    """

    accelerations = compute_accelerations(
        particles, theta=theta, G=G, eps=eps
    )

    acc_magnitudes = np.linalg.norm(accelerations, axis=1)
    max_acc = np.max(acc_magnitudes)

    if max_acc == 0:
        return dt_max

    # Adaptive timestep estimate
    dt = eta * np.sqrt(eps / max_acc)

    # Prevent dt from becoming too small or too large, this was suggested by
    ↪ Gen-AI when I ran into a long/infinite runtime issue
    dt = np.clip(dt, dt_min, dt_max)

    return dt

def evolve_adaptive_timestep(
    particles,
    t_end,
    theta=0.5,
    G=1.0,
    eps=0.05,
    dt_max=0.02,
    dt_min=0.002,
    eta=0.2,
    save_every=5,
    max_steps=5000
):
    """
    Evolve the system using RK4 with adaptive timestep.

    Includes max_steps so the code cannot run forever.
    """

    times = []

```



```

history = []
dt_history = []

t = 0.0
step = 0

while t < t_end and step < max_steps:

    if step % save_every == 0:
        times.append(t)
        history.append(np.array([p.position.copy() for p in particles]))

    dt = choose_adaptive_dt(
        particles,
        dt_max=dt_max,
        dt_min=dt_min,
        eta=eta,
        theta=theta,
        G=G,
        eps=eps
    )

    dt = min(dt, t_end - t)
    dt_history.append(dt)

    rk4_step(particles, dt, theta=theta, G=G, eps=eps)

    t += dt
    step += 1

    # Print progress occasionally
    if step % 100 == 0:
        print(f"Step {step}, time = {t:.4f}, dt = {dt:.5f}")

    if step >= max_steps:
        print("Warning: reached max_steps before t_end.")

    return np.array(times), np.array(history), np.array(dt_history)

```

0.0.4 Breakdown of code cell above

This cell is organized into the following pieces:

- Defines reusable functions used by later cells:
 - `compute_force_from_node` evaluates the Barnes-Hut force contribution from a tree node.
 - `compute_accelerations` builds a tree and computes accelerations for all particles.
 - `get_state` converts particle objects into NumPy arrays.

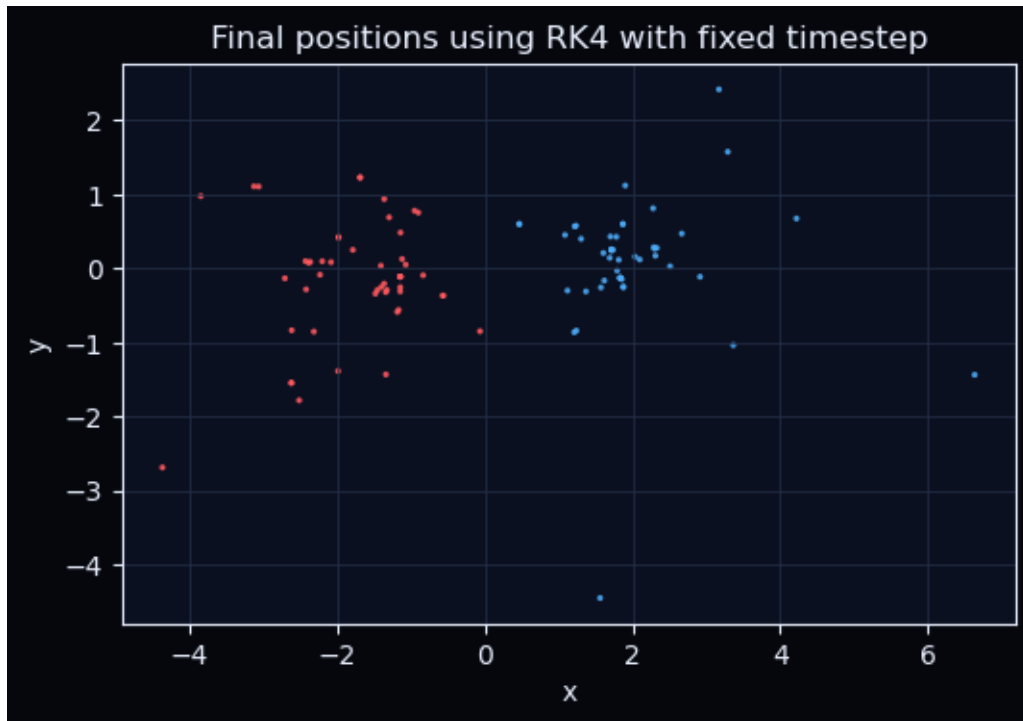
- `set_state` writes NumPy state arrays back into particle objects.
- `temporary_particles` creates temporary particle objects for intermediate RK4 states.
- `derivatives` returns position and velocity derivatives for RK4.
- `rk4_step` advances the system by one fourth-order Runge-Kutta step.
- `evolve_fixed_timestep` runs the RK4 solver with a constant timestep.
- `choose_adaptive_dt` selects a timestep from the maximum acceleration.
- `evolve_adaptive_timestep` runs RK4 with a timestep that changes during the simulation.
- Evolves the particle system with fixed-timestep RK4.

```
[9]: # Initialize two galaxies.
particles_fixed, theta = initialize_two_galaxies(
    N_per_galaxy=50,
    d=5.0,
    theta=0.5,
    v_offset=0.2
)

# Evolve using fixed timestep RK4.
times_fixed, history_fixed = evolve_fixed_timestep(
    particles_fixed,
    t_end=2.0,
    dt=0.01,
    theta=theta,
    G=1.0,
    eps=0.01,
    save_every=5
)

# Plot final particle positions.
final_positions = history_fixed[-1]

fig, ax = plt.subplots(figsize=(6, 6))
scatter_galaxy_positions(ax, final_positions, s=5)
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("Final positions using RK4 with fixed timestep")
ax.set_aspect("equal")
style_dark_axis(ax)
plt.show()
```



0.0.5 Breakdown of code cell above

This cell is organized into the following pieces:

- Sets up two Plummer-like galaxies separated in space and gives them bulk velocities toward one another.
- Evolves the particle system with fixed-timestep RK4.
- Builds Matplotlib figures for visual comparison or diagnostic plots.

Clearly not running for long enough

```
[10]: # Lets investigate the adaptive timestep choice
particles_adaptive, theta = initialize_two_galaxies(
    N_per_galaxy=50,
    d=5.0,
    theta=0.5,
    v_offset=0.2
)

times_adaptive, history_adaptive, dt_history = evolve_adaptive_timestep(
    particles_adaptive,
    t_end=0.5,
    theta=theta,
    G=1.0,
    eps=0.05,
```

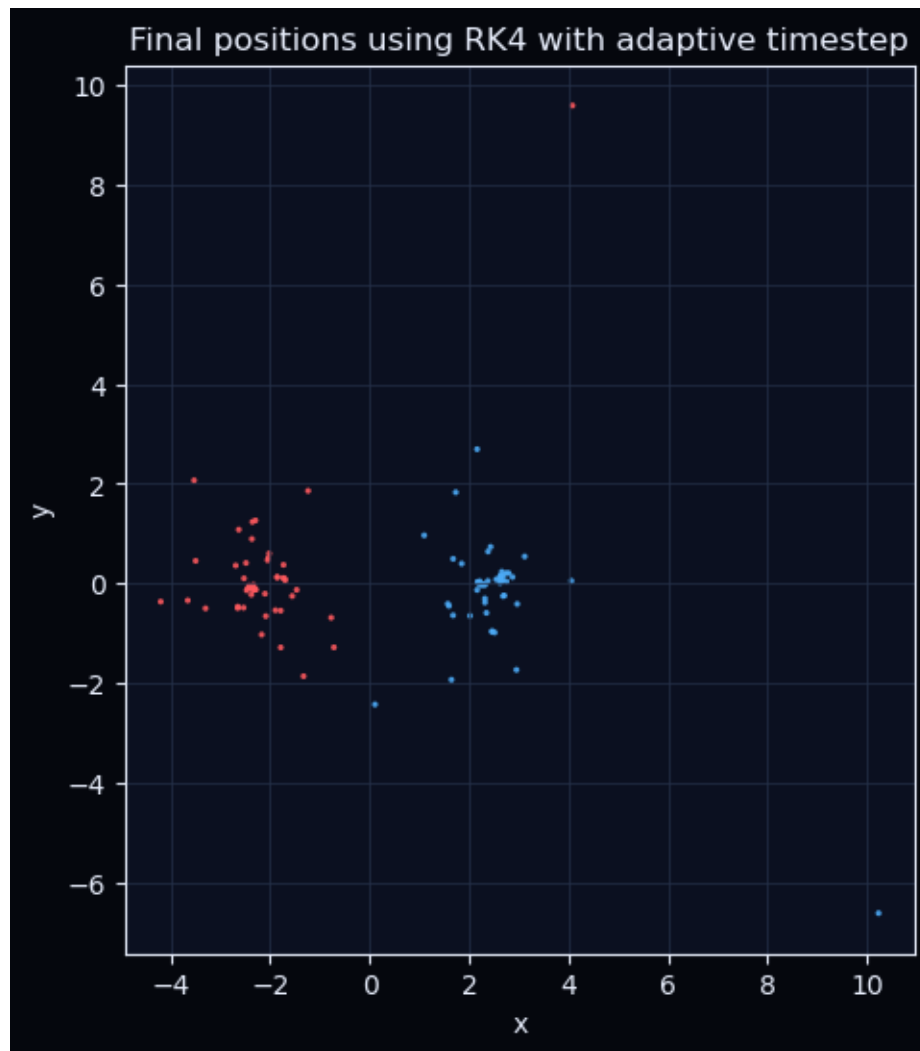
```

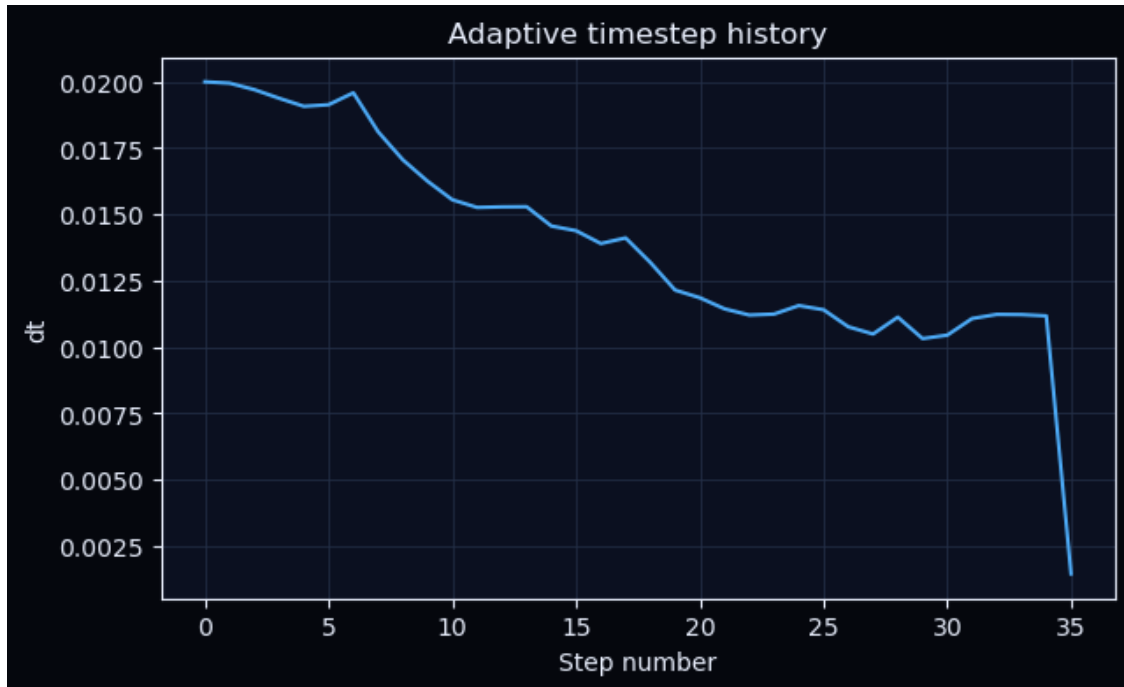
    dt_max=0.02,
    dt_min=0.002,
    eta=0.2,
    save_every=5,
    max_steps=5000
)
final_positions = history_adaptive[-1]

fig, ax = plt.subplots(figsize=(6, 6))
scatter_galaxy_positions(ax, final_positions, s=5)
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("Final positions using RK4 with adaptive timestep")
ax.set_aspect("equal")
style_dark_axis(ax)
plt.show()

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(dt_history, color=GALAXY_2_COLOR)
ax.set_xlabel("Step number")
ax.set_ylabel("dt")
ax.set_title("Adaptive timestep history")
style_dark_axis(ax)
plt.show()

```





0.0.6 Breakdown of code cell above

This cell is organized into the following pieces:

- Sets up two Plummer-like galaxies separated in space and gives them bulk velocities toward one another.
- Evolves the particle system with an adaptive timestep chosen from the maximum acceleration.

```
[11]: # Sanity checks and visualization tools

def check_particle_data(particles):
    """
    Sanity check 1:
    Make sure all particles have valid positions, velocities, and masses.
    """

    print("Running particle data sanity check...")

    for i, p in enumerate(particles):

        # Check that position has two components
        if len(p.position) != 2:
            raise ValueError(f"Particle {i} does not have a 2D position.")

        # Check that velocity has two components
        if len(p.velocity) != 2:
```

```

        raise ValueError(f"Particle {i} does not have a 2D velocity.")

    # Check for NaN or infinite position values
    if not np.all(np.isfinite(p.position)):
        raise ValueError(f"Particle {i} has invalid position: {p.position}")

    # Check for NaN or infinite velocity values
    if not np.all(np.isfinite(p.velocity)):
        raise ValueError(f"Particle {i} has invalid velocity: {p.velocity}")

    # Check that mass is positive and finite
    if not np.isfinite(p.mass) or p.mass <= 0:
        raise ValueError(f"Particle {i} has invalid mass: {p.mass}")

print("Passed: all particles have valid data.")

def check_total_mass(particles, expected_mass=None):
    """
    Sanity check 2:
    Check that the total particle mass is sensible.

    For one galaxy initialized with initialize_particles(N),
    total mass should be approximately 1.

    For two galaxies initialized with initialize_two_galaxies(N),
    total mass should be approximately 2.
    """

    total_mass = sum(p.mass for p in particles)

    print("Running total mass sanity check...")
    print(f"Total mass = {total_mass:.6f}")

    if expected_mass is not None:
        print(f"Expected mass = {expected_mass:.6f}")

        if not np.isclose(total_mass, expected_mass):
            raise ValueError("Total mass does not match expected mass.")

    print("Passed: total mass looks sensible.")

def check_center_of_mass(particles):
    """
    Sanity check 3:
    Compute and print the center of mass of the full particle system.

```

```

"""

total_mass = sum(p.mass for p in particles)

center_of_mass = sum(
    p.mass * p.position for p in particles
) / total_mass

print("Running center of mass sanity check...")
print(f"Center of mass = {center_of_mass}")

if not np.all(np.isfinite(center_of_mass)):
    raise ValueError("Center of mass is invalid.")

print("Passed: center of mass is finite.")

return center_of_mass

def check_quadtree_mass(particles, theta=0.5):
    """
    Sanity check 4:
    Build a Barnes-Hut tree and check that the root node contains
    the same total mass as the particle list.
    """

    print("Running quadtree mass sanity check...")

    tree = QuadTree(particles, theta)

    particle_mass = sum(p.mass for p in particles)
    tree_mass = tree.root.total_mass

    print(f"Particle mass total = {particle_mass:.6f}")
    print(f"Tree root mass      = {tree_mass:.6f}")

    if not np.isclose(particle_mass, tree_mass):
        raise ValueError("Tree root mass does not match particle total mass.")

    print("Passed: quadtree root mass matches particle mass.")

def check_accelerations(particles, theta=0.5, G=1.0, eps=0.05):
    """
    Sanity check 5:
    Compute accelerations and check that they are finite.

```



```

This catches numerical problems such as NaN or infinite forces.
"""

print("Running acceleration sanity check...")

accelerations = compute_accelerations(
    particles,
    theta=theta,
    G=G,
    eps=eps
)

if not np.all(np.isfinite(accelerations)):
    raise ValueError("Some accelerations are NaN or infinite.")

max_acc = np.max(np.linalg.norm(accelerations, axis=1))

print(f"Maximum acceleration magnitude = {max_acc:.6f}")
print("Passed: accelerations are finite.")

return accelerations

def run_all_sanity_checks(particles, theta=0.5, expected_mass=None):
    """
    Run all basic sanity checks together.
    """

    print("=====")
    print("Running all sanity checks")
    print("=====")

    check_particle_data(particles)
    check_total_mass(particles, expected_mass=expected_mass)
    check_center_of_mass(particles)
    check_quadtree_mass(particles, theta=theta)
    check_accelerations(particles, theta=theta)

    print("=====")
    print("All sanity checks passed.")
    print("=====")

def plot_particles(particles, title="Particle positions", s=8):
    """
    Visualization:
    Plot the current particle positions.

```

```

"""

fig, ax = plt.subplots(figsize=(6, 6))
scatter_particle_list(ax, particles, s=s)
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title(title)
ax.set_aspect("equal")
style_dark_axis(ax)
plt.show()

def plot_simulation_frame(history, frame=-1, title=None, s=8):
    """
    Visualization:
    Plot one saved frame from a simulation history.

    history has shape:
        number_of_frames, number_of_particles, 2
    """

    positions = history[frame]

    if title is None:
        title = f"Simulation frame {frame}"

    fig, ax = plt.subplots(figsize=(6, 6))
    scatter_galaxy_positions(ax, positions, s=s)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_title(title)
    ax.set_aspect("equal")
    style_dark_axis(ax)
    plt.show()

def plot_simulation_snapshots(history, times=None, frames=None, s=5):
    """
    Visualization:
    Plot several simulation snapshots side by side.

    This is useful for showing the time evolution of the galaxy merger.
    """

    if frames is None:
        frames = [0, len(history)//3, 2*len(history)//3, len(history)-1]

```

```

fig, axes = plt.subplots(1, len(frames), figsize=(5 * len(frames), 5))
fig.patch.set_facecolor(DARK_BACKGROUND)

if len(frames) == 1:
    axes = [axes]

for ax, frame in zip(axes, frames):

    positions = history[frame]

    scatter_galaxy_positions(ax, positions, s=s)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_aspect("equal")
    style_dark_axis(ax)

    if times is not None:
        ax.set_title(f"t = {times[frame]:.3f}")
    else:
        ax.set_title(f"Frame {frame}")

plt.tight_layout()
plt.show()

# The following code was written in bulk by Gen-AI to help simulate my results.
# I attempted it first but then optimised and made better,
# using AI
def animate_simulation(
    history,
    times=None,
    interval=50,
    s=5,
    zoom=None
):
    """
    Animate the simulation history with optional zoom.

    Parameters
    -----
    history : ndarray
        Saved particle positions.

    times : ndarray
        Saved simulation times.

```

```

interval : int
    Delay between animation frames in milliseconds.

s : float
    Marker size.

zoom : float or None
    If None:
        Automatically scale to include all particles.

    If a number:
        Set plot limits to:
            x = [-zoom, +zoom]
            y = [-zoom, +zoom]

    This lets you zoom into the merger region.
"""

from matplotlib.animation import FuncAnimation
from IPython.display import HTML

fig, ax = plt.subplots(figsize=(6, 6))
fig.patch.set_facecolor(DARK_BACKGROUND)

# -----
# Automatic full-domain scaling
# -----
if zoom is None:

    all_x = history[:, :, 0]
    all_y = history[:, :, 1]

    x_min, x_max = np.min(all_x), np.max(all_x)
    y_min, y_max = np.min(all_y), np.max(all_y)

    padding = 0.1 * max(x_max - x_min, y_max - y_min)

    ax.set_xlim(x_min - padding, x_max + padding)
    ax.set_ylim(y_min - padding, y_max + padding)

# -----
# User-controlled zoom
# -----
else:

    ax.set_xlim(-zoom, zoom)
    ax.set_ylim(-zoom, zoom)

```

```

ax.set_aspect("equal")
ax.set_xlabel("x")
ax.set_ylabel("y")
style_dark_axis(ax)

scatter = scatter_galaxy_positions(ax, history[0], s=s)

def update(frame):

    positions = history[frame]

    scatter.set_offsets(positions)

    if times is not None:
        ax.set_title(
            f"Galaxy merger simulation, t = {times[frame]:.3f}"
        )
    else:
        ax.set_title(
            f"Galaxy merger simulation, frame = {frame}"
        )

    return scatter,

anim = FuncAnimation(
    # Create or configure the Matplotlib figure.
    fig,
    update,
    frames=len(history),
    interval=interval,
    blit=True
)

plt.close(fig)

return HTML(anim.to_jshtml())

```

0.0.7 Breakdown of code cell above

This cell is organized into the following pieces:

- Imports the main numerical, plotting, statistics, and astronomy libraries used later in the notebook: IPython, matplotlib.
- Defines reusable functions used by later cells:
 - `check_particle_data` supports the simulation workflow.
 - `check_total_mass` supports the simulation workflow.
 - `check_center_of_mass` supports the simulation workflow.

- `check_quadtree_mass` supports the simulation workflow.
- `check_accelerations` supports the simulation workflow.
- `run_all_sanity_checks` runs the validation checks together.
- `plot_particles` plots a single particle distribution.
- `plot_simulation_frame` plots one stored simulation frame.
- `plot_simulation_snapshots` plots several stored frames side by side.
- `animate_simulation` creates a notebook animation from stored positions.
- Sets up two Plummer-like galaxies separated in space and gives them bulk velocities toward one another.
- Plots stored snapshots so the merger morphology can be compared at several times.
- Creates an animation from the stored position history.
- Builds Matplotlib figures for visual comparison or diagnostic plots.
- Performs sanity checks to verify masses, positions, tree mass, and accelerations.

```
[12]: # Run this and see what we get
particles_test, theta = initialize_two_galaxies(
    N_per_galaxy=50,
    d=5.0,
    theta=0.5,
    v_offset=0.2
)

run_all_sanity_checks(
    particles_test,
    theta=theta,
    expected_mass=2.0
)

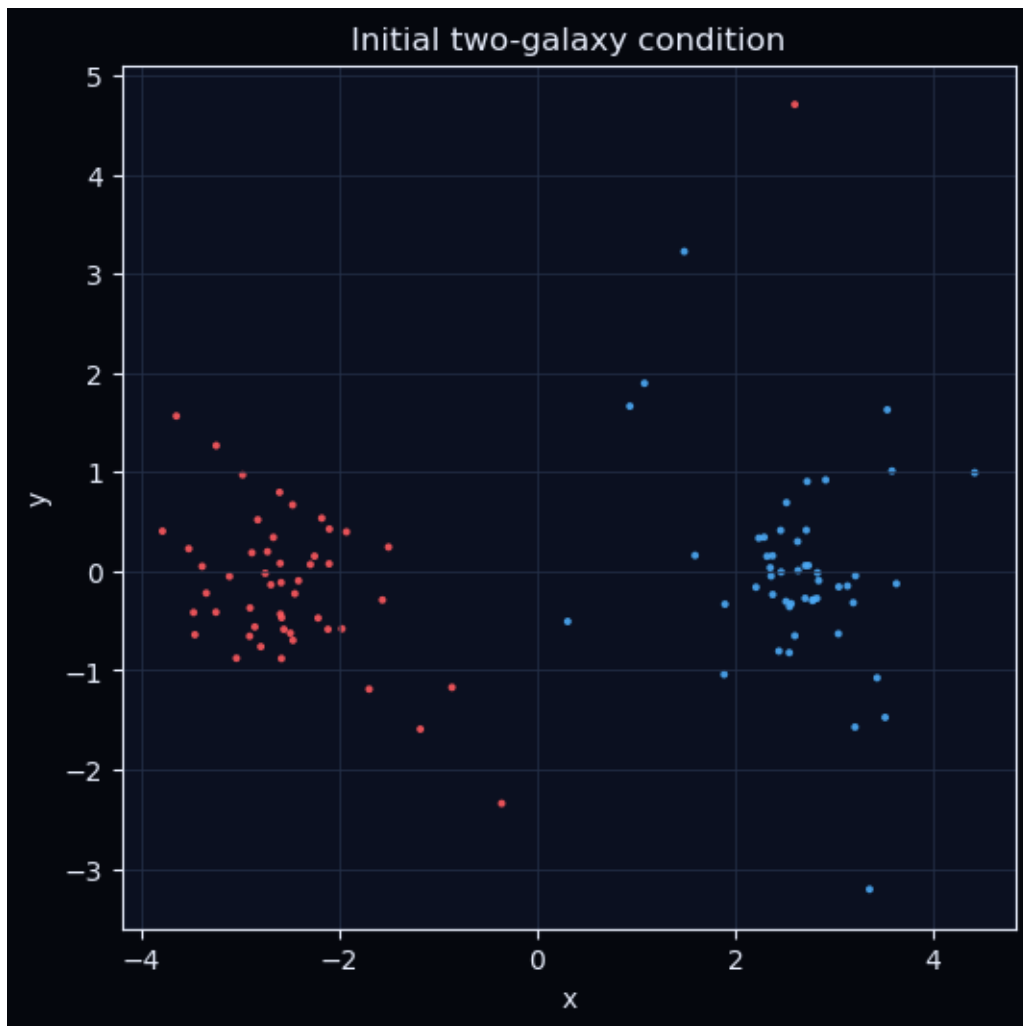
plot_particles(
    particles_test,
    title="Initial two-galaxy condition",
    s=8
)
```

```
=====
Running all sanity checks
=====
Running particle data sanity check...
Passed: all particles have valid data.
Running total mass sanity check...
Total mass = 2.000000
Expected mass = 2.000000
Passed: total mass looks sensible.
Running center of mass sanity check...
Center of mass = [ 0.0818171 -0.03874006]
Passed: center of mass is finite.
Running quadtree mass sanity check...
Particle mass total = 2.000000
```

```

Tree root mass      = 2.000000
Passed: quadtree root mass matches particle mass.
Running acceleration sanity check...
Maximum acceleration magnitude = 7.339771
Passed: accelerations are finite.
=====
All sanity checks passed.
=====

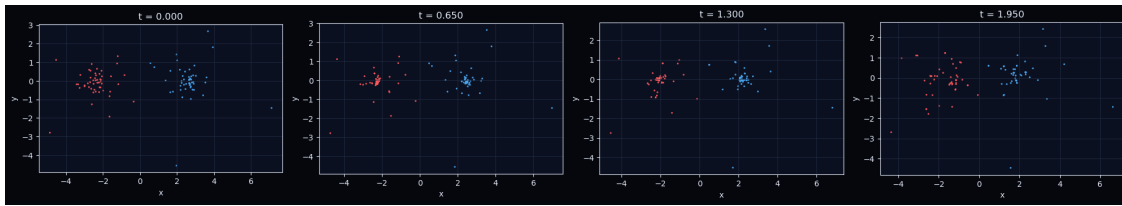
```



```

[13]: # Lets plot these snapshots as a vibe check
      plot_simulation_snapshots(
        history_fixed,
        times=times_fixed,
        s=5
      )

```



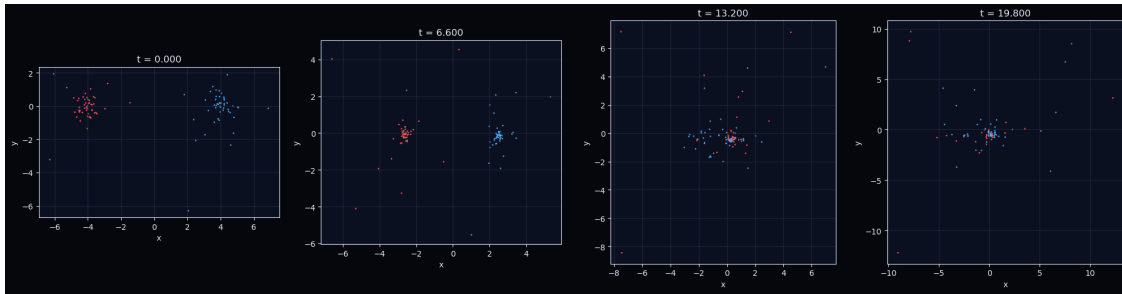
```
[14]: # The snapshots seemed reasonable so now lets animate it and see what we are
      ↪ working with
animate_simulation(
    history_fixed,
    times=times_fixed,
    interval=80,
    s=5
)
```

```
[14]: <IPython.core.display.HTML object>
```

```
[15]: # Lets see if we can get it to be a proper merger so lets run for longer and
      ↪ start with different distance etc
particles_merger, theta = initialize_two_galaxies(
    N_per_galaxy=50,
    d=8.0,          # smaller separation than 5.0
    theta=0.5,
    v_offset=0.15   # slower approach than 0.2
)

times_merger, history_merger = evolve_fixed_timestep(
    particles_merger,
    t_end=20.0,     # run much longer
    dt=0.01,
    theta=theta,
    G=1.0,
    eps=0.05,
    save_every=20
)

plot_simulation_snapshots(
    history_merger,
    times=times_merger,
    s=4
)
```

```
[16]: animate_simulation(
        history_merger,
        times=times_merger,
        interval=80,
        s=4
    )
```

[16]: <IPython.core.display.HTML object>

[]:

0.1 Optimisation: vectorized direct summation for small particle numbers

0.1.1 This was a suggestion given during a tutorial and with discussion with peers in course but was implemented largely with gen-AI as it is just to cut down the runtimes of my code so I can explore the problem in more depth without having to wait 1hr+ each time to see the simulations and results

The Barnes-Hut implementation above is the required algorithm for the project, but for small systems the Python quadtree overhead can be larger than the cost of an exact, vectorized direct-summation force calculation. The cells below keep the original Barnes-Hut force routine available, add a direct NumPy force calculation, and compare both methods on identical initial conditions.

```
[17]: # =====
# Optimisation: direct summation for small-N simulations
# =====

# Keep a reference to the Barnes-Hut acceleration function defined earlier.
# This lets us compare the original method against the optimised direct method.
compute_accelerations_barnes_hut = compute_accelerations

def compute_accelerations_direct_arrays(positions, masses, G=1.0, eps=0.01):
    """
    Compute all particle accelerations using vectorized direct summation.

    This is still  $O(N^2)$ , but the work is done inside NumPy instead of Python
    """
```

loops and recursive tree traversal. For small N , this can be faster than the pure-Python Barnes-Hut implementation.

Parameters

positions : ndarray, shape (N, 2)
Particle positions.

masses : ndarray, shape (N,)
Particle masses.

G : float
Gravitational constant in code units.

eps : float
Softening length.

"""

diff[i, j] = r_j - r_i
diff = positions[None, :, :] - positions[:, None, :]

Softened squared distance.
r2 = np.sum(diff**2, axis=2) + eps**2

Compute 1 / r^3 for every pair.
inv_r3 = r2**(-1.5)

Remove self-force terms.
np.fill_diagonal(inv_r3, 0.0)

*# a_i = G * sum_j m_j * (r_j - r_i) / r^3*
accelerations = G * np.sum(
 diff * masses[None, :, None] * inv_r3[:, :, None],
 axis=1
)

return accelerations

def compute_accelerations_direct(particles, G=1.0, eps=0.01):

"""

Particle-object wrapper around the vectorized direct acceleration routine.
"""

positions, velocities, masses = get_state(particles)
return compute_accelerations_direct_arrays(
 positions,

```

        masses,
        G=G,
        eps=eps
    )

def compute_accelerations_auto(
    particles,
    theta=0.5,
    G=1.0,
    eps=0.01,
    method="auto",
    direct_threshold=300
):
    """
    Choose the acceleration method.

    method="direct" uses vectorized direct summation.
    method="barnes-hut" uses the original Barnes-Hut routine.
    method="auto" uses direct summation below direct_threshold and
    Barnes-Hut above it.
    """

    N = len(particles)

    if method == "direct":
        return compute_accelerations_direct(particles, G=G, eps=eps)

    if method == "barnes-hut":
        return compute_accelerations_barnes_hut(
            particles,
            theta=theta,
            G=G,
            eps=eps
        )

    if method == "auto":
        if N <= direct_threshold:
            return compute_accelerations_direct(particles, G=G, eps=eps)

        return compute_accelerations_barnes_hut(
            particles,
            theta=theta,
            G=G,
            eps=eps
        )

```

```

raise ValueError("method must be 'auto', 'direct', or 'barnes-hut'.")

def derivatives_selectable(
    positions,
    velocities,
    masses,
    theta=0.5,
    G=1.0,
    eps=0.01,
    method="auto",
    direct_threshold=300
):
    """
    RK4 derivatives using a selectable force calculation method.

    For the direct method we avoid creating temporary Particle objects, which
    gives an extra speed improvement during RK4 substeps.
    """

    dpos_dt = velocities

    use_direct = (
        method == "direct"
        or (method == "auto" and len(masses) <= direct_threshold)
    )

    if use_direct:
        dvel_dt = compute_accelerations_direct_arrays(
            positions,
            masses,
            G=G,
            eps=eps
        )
    else:
        temp = temporary_particles(positions, velocities, masses)
        dvel_dt = compute_accelerations_barnes_hut(
            temp,
            theta=theta,
            G=G,
            eps=eps
        )

    return dpos_dt, dvel_dt

def rk4_step_selectable(

```

```

particles,
dt,
theta=0.5,
G=1.0,
eps=0.01,
method="auto",
direct_threshold=300
):
    """
    RK4 step with selectable Barnes-Hut/direct acceleration calculation.
    """

    pos, vel, masses = get_state(particles)

    k1_pos, k1_vel = derivatives_selectable(
        pos,
        vel,
        masses,
        theta=theta,
        G=G,
        eps=eps,
        method=method,
        direct_threshold=direct_threshold
    )

    k2_pos, k2_vel = derivatives_selectable(
        pos + 0.5 * dt * k1_pos,
        vel + 0.5 * dt * k1_vel,
        masses,
        theta=theta,
        G=G,
        eps=eps,
        method=method,
        direct_threshold=direct_threshold
    )

    k3_pos, k3_vel = derivatives_selectable(
        pos + 0.5 * dt * k2_pos,
        vel + 0.5 * dt * k2_vel,
        masses,
        theta=theta,
        G=G,
        eps=eps,
        method=method,
        direct_threshold=direct_threshold
    )

```

```

k4_pos, k4_vel = derivatives_selectable(
    pos + dt * k3_pos,
    vel + dt * k3_vel,
    masses,
    theta=theta,
    G=G,
    eps=eps,
    method=method,
    direct_threshold=direct_threshold
)

new_pos = pos + (dt / 6.0) * (
    k1_pos + 2.0 * k2_pos + 2.0 * k3_pos + k4_pos
)

new_vel = vel + (dt / 6.0) * (
    k1_vel + 2.0 * k2_vel + 2.0 * k3_vel + k4_vel
)

set_state(particles, new_pos, new_vel)

def evolve_fixed_timestep_selectable(
    particles,
    t_end,
    dt,
    theta=0.5,
    G=1.0,
    eps=0.01,
    save_every=1,
    method="auto",
    direct_threshold=300
):
    """
    Fixed-timestep RK4 evolution with selectable force calculation.
    """

    times = []
    history = []

    t = 0.0
    step = 0

    while t < t_end:
        if step % save_every == 0:
            times.append(t)
            history.append(np.array([p.position.copy() for p in particles]))

```

```

        current_dt = min(dt, t_end - t)

        rk4_step_selectable(
            particles,
            current_dt,
            theta=theta,
            G=G,
            eps=eps,
            method=method,
            direct_threshold=direct_threshold
        )

        t += current_dt
        step += 1

    return np.array(times), np.array(history)

def clone_particles(particles):
    """
    Make an independent copy of a particle list.
    """

    return [
        Particle(p.position.copy(), p.velocity.copy(), p.mass)
        for p in particles
    ]

```

0.1.2 Breakdown of code cell above

This cell is organized into the following pieces:

- Defines reusable functions used by later cells:
 - `compute_accelerations_direct_arrays` computes direct-summation accelerations using vectorized NumPy arrays.
 - `compute_accelerations_direct` wraps the direct array method for particle objects.
 - `compute_accelerations_auto` chooses direct summation or Barnes-Hut based on method settings.
 - `derivatives_selectable` computes RK4 derivatives using the selected force method.
 - `rk4_step_selectable` performs one RK4 step with selectable force calculation.
 - `evolve_fixed_timestep_selectable` runs fixed-timestep RK4 with direct/Barnes-Hut/auto force selection.
 - `clone_particles` copies particle lists so runs can start from identical initial conditions.
- Evolves the system with the optimized selectable force calculation, using direct summation for small particle numbers.

In the full notebook sequence, this cell either builds part of the N-body machinery, runs a specific simulation, or analyzes the resulting trajectories and energy behavior.

```

[18]: # =====
# Timing comparison: original Barnes-Hut vs vectorized direct
# =====

import time

# Use a fixed random seed so both methods start from exactly the same system.
# Set the random seed for reproducible initial conditions.
np.random.seed(7)

initial_particles, theta_compare = initialize_two_galaxies(
    N_per_galaxy=50,
    d=5.0,
    theta=0.5,
    v_offset=0.2
)

particles_bh_compare = clone_particles(initial_particles)
particles_direct_compare = clone_particles(initial_particles)

benchmark_t_end = 2.0
benchmark_dt = 0.01
benchmark_eps = 0.05

start = time.perf_counter()
times_bh_compare, history_bh_compare = evolve_fixed_timestep_selectable(
    particles_bh_compare,
    t_end=benchmark_t_end,
    dt=benchmark_dt,
    theta=theta_compare,
    G=1.0,
    eps=benchmark_eps,
    save_every=10,
    method="barnes-hut"
)
runtime_bh = time.perf_counter() - start

start = time.perf_counter()
times_direct_compare, history_direct_compare = evolve_fixed_timestep_selectable(
    particles_direct_compare,
    t_end=benchmark_t_end,
    dt=benchmark_dt,
    theta=theta_compare,
    G=1.0,
    eps=benchmark_eps,
    save_every=10,
    method="direct"
)

```



```

)
runtime_direct = time.perf_counter() - start

print(f"Barnes-Hut runtime:           {runtime_bh:.3f} s")
print(f"Vectorized direct runtime:    {runtime_direct:.3f} s")
print(f"Speed-up factor:               {runtime_bh / runtime_direct:.2f}x")

# Create or configure the Matplotlib figure.
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
fig.patch.set_facecolor(DARK_BACKGROUND)

axes[0].bar(
    ["Barnes-Hut", "Direct"],
    [runtime_bh, runtime_direct],
    color=["tab:blue", "tab:orange"]
)
axes[0].set_ylabel("Runtime [s]")
axes[0].set_title("Runtime comparison")
style_dark_axis(axes[0])

scatter_galaxy_positions(axes[1], history_bh_compare[-1], s=5)
axes[1].set_title("Barnes-Hut final state")
axes[1].set_aspect("equal")
style_dark_axis(axes[1])

scatter_galaxy_positions(axes[2], history_direct_compare[-1], s=5)
axes[2].set_title("Direct final state")
axes[2].set_aspect("equal")
style_dark_axis(axes[2])

for ax in axes[1:]:
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    style_dark_axis(ax)

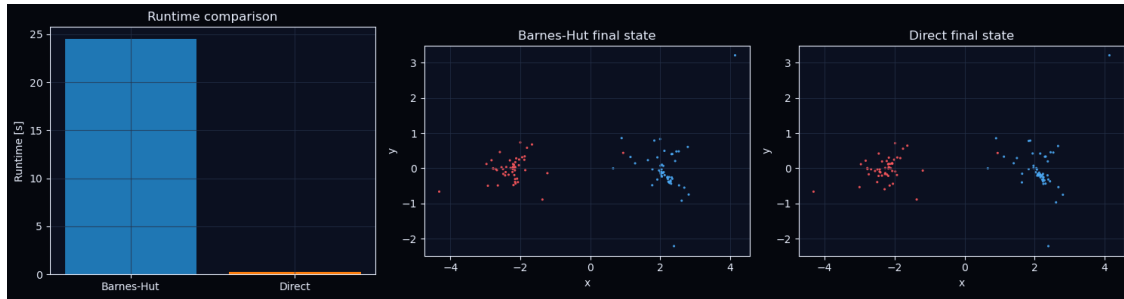
plt.tight_layout()
plt.show()

```

```

Barnes-Hut runtime:           24.523 s
Vectorized direct runtime:    0.278 s
Speed-up factor:             88.12x

```



0.1.3 Breakdown of code cell above

This cell is organized into the following pieces:

- Imports the main numerical, plotting, statistics, and astronomy libraries used later in the notebook: `time`.
- Sets up two Plummer-like galaxies separated in space and gives them bulk velocities toward one another.
- Evolves the system with the optimized selectable force calculation, using direct summation for small particle numbers.
- Builds Matplotlib figures for visual comparison or diagnostic plots.
- Measures runtime so different numerical methods can be compared quantitatively.

```
[19]: # Optimised long run using automatic method selection and hopefully much faster

# For N_per_galaxy=50, method="auto" selects vectorized direct summation
# For larger simulations above direct_threshold, it switches back to Barnes-Hut

# Set the random seed for reproducible initial conditions
np.random.seed(11)

particles_optimised_long, theta_optimised = initialize_two_galaxies(
    N_per_galaxy=150,
    d=20.0,
    theta=0.5,
    v_offset=0.2
)

start = time.perf_counter()
times_optimised_long, history_optimised_long = evolve_fixed_timestep_selectable(
    particles_optimised_long,
    t_end=50.0,
    dt=0.01,
    theta=theta_optimised,
    G=1.0,
```

```

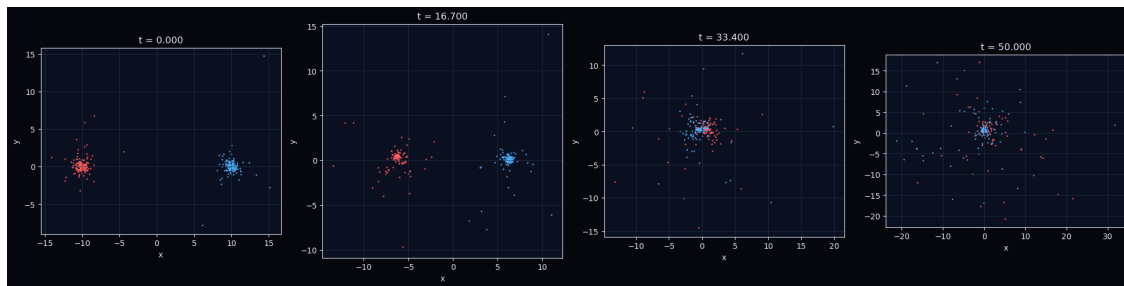
    eps=0.05,
    save_every=10,
    method="auto",
    direct_threshold=500
)
runtime_optimised_long = time.perf_counter() - start

print(f"Optimised long-run runtime: {runtime_optimised_long:.3f} s")

plot_simulation_snapshots(
    history_optimised_long,
    times=times_optimised_long,
    s=4
)

```

Optimised long-run runtime: 60.476 s



0.1.4 Breakdown of code cell above

This cell is organized into the following pieces:

- Sets up two Plummer-like galaxies separated in space and gives them bulk velocities toward one another.
- Evolves the system with the optimized selectable force calculation, using direct summation for small particle numbers.
- Plots stored snapshots so the merger morphology can be compared at several times.
- Measures runtime so different numerical methods can be compared quantitatively.

```

[20]: animate_simulation(
    history_optimised_long,
    times=times_optimised_long,
    interval=80,
    s=4
)

```

[20]: <IPython.core.display.HTML object>

[]:

0.2 Total energy over time

The next cell measures the kinetic, potential, and total energy of the system during an optimised RK4 run. The potential energy uses the same softened distance convention as the force calculation, so the energy diagnostic is consistent with the simulated dynamics.

```
[21]: # Energy conservation diagnostic lets do this !!!!

def total_energy_arrays(positions, velocities, masses, G=1.0, eps=0.05):
    """
    Compute the total energy of the N-body system.

    The total energy is:

        
$$E = K + U$$


    where

        
$$K = 1/2 \sum_i m_i |v_i|^2$$


    and

        
$$U = -G \sum_{\{i < j\}} m_i m_j / \sqrt{(|r_i - r_j|^2 + \text{eps}^2)}$$


    The softened potential energy is chosen to match the softened force law
    used earlier in the notebook.
    """

    positions = np.asarray(positions)
    velocities = np.asarray(velocities)
    masses = np.asarray(masses)

    # Kinetic energy
    speeds_squared = np.sum(velocities**2, axis=1)
    kinetic_energy = 0.5 * np.sum(masses * speeds_squared)

    # Pairwise position differences
    # diff[i, j] = r_i - r_j
    diff = positions[:, None, :] - positions[None, :, :]

    # Softened pairwise distances
    distances = np.sqrt(np.sum(diff**2, axis=2) + eps**2)

    # Use only i < j pairs so each particle pair is counted once
```

```

i_upper, j_upper = np.triu_indices(len(masses), k=1)

potential_energy = -G * np.sum(
    masses[i_upper] * masses[j_upper] / distances[i_upper, j_upper]
)

total_energy = kinetic_energy + potential_energy

return total_energy, kinetic_energy, potential_energy

def total_energy_particles(particles, G=1.0, eps=0.05):
    """
    Convenience wrapper that computes total energy from a list of Particle_
    ↪objects.
    """

    positions, velocities, masses = get_state(particles)
    return total_energy_arrays(
        positions,
        velocities,
        masses,
        G=G,
        eps=eps
    )

def evolve_fixed_timestep_selectable_with_energy(
    particles,
    t_end,
    dt,
    theta=0.5,
    G=1.0,
    eps=0.05,
    save_every=1,
    method="auto",
    direct_threshold=300
):
    """
    Evolve the system with the optimised/selectable RK4 solver while recording_
    ↪energy.

    This is very similar to evolve_fixed_timestep_selectable, but it also saves:
    - velocities
    - kinetic energy
    - potential energy
    - total energy

```

Saving velocities is important because total energy cannot be computed properly

from the position-only history used for animations.
"""

```
times = []
position_history = []
velocity_history = []
kinetic_history = []
potential_history = []
total_energy_history = []

t = 0.0
step = 0

def save_current_state(current_time):
    positions, velocities, masses = get_state(particles)
    total_E, kinetic_E, potential_E = total_energy_arrays(
        positions,
        velocities,
        masses,
        G=G,
        eps=eps
    )

    times.append(current_time)
    position_history.append(positions.copy())
    velocity_history.append(velocities.copy())
    total_energy_history.append(total_E)
    kinetic_history.append(kinetic_E)
    potential_history.append(potential_E)

while t < t_end:
    if step % save_every == 0:
        save_current_state(t)

    current_dt = min(dt, t_end - t)

    rk4_step_selectable(
        particles,
        current_dt,
        theta=theta,
        G=G,
        eps=eps,
        method=method,
        direct_threshold=direct_threshold
```

```

    )

    t += current_dt
    step += 1

    # Save the exact final state as well, even if it does not land on save_every
    if len(times) == 0 or not np.isclose(times[-1], t_end):
        save_current_state(t_end)

    return (
        np.array(times),
        np.array(position_history),
        np.array(velocity_history),
        np.array(total_energy_history),
        np.array(kinetic_history),
        np.array(potential_history)
    )

# Run an optimised RK4 simulation and plot energy over time.

# Set the random seed for reproducible initial conditions
np.random.seed(23)

particles_energy, theta_energy = initialize_two_galaxies(
    N_per_galaxy=100,
    d=20.0,
    theta=0.5,
    v_offset=0.15
)

energy_t_end = 400.0 # Much longer time as want to make the energy drift more
    ↳ apperent
energy_dt = 0.01
energy_eps = 0.05

start = time.perf_counter()
(
    times_energy,
    history_energy,
    velocity_history_energy,
    total_energy_history,
    kinetic_energy_history,
    potential_energy_history
) = evolve_fixed_timestep_selectable_with_energy(

```

```

    particles_energy,
    t_end=energy_t_end,
    dt=energy_dt,
    theta=theta_energy,
    G=1.0,
    eps=energy_eps,
    save_every=10,
    method="auto",
    direct_threshold=600
)
runtime_energy = time.perf_counter() - start

# Fractional energy change relative to the initial energy
initial_energy = total_energy_history[0]
relative_energy_change = (
    total_energy_history - initial_energy
) / abs(initial_energy)

final_drift = relative_energy_change[-1]
max_drift = np.max(np.abs(relative_energy_change))

print(f"Energy run runtime: {runtime_energy:.3f} s")
print(f"Initial total energy: {initial_energy:.6f}")
print(f"Final total energy: {total_energy_history[-1]:.6f}")
print(f"Final fractional energy drift: {final_drift:.3e}")
print(f"Maximum absolute fractional drift: {max_drift:.3e}")

# Plot kinetic, potential, and total energy

# Create or configure the Matplotlib figure.
fig, axes = plt.subplots(1, 2, figsize=(13, 4))

axes[0].plot(times_energy, kinetic_energy_history, label="Kinetic energy")
axes[0].plot(times_energy, potential_energy_history, label="Potential energy")
axes[0].plot(times_energy, total_energy_history, label="Total energy",
    ↪linewidth=2)
axes[0].set_xlabel("Time")
axes[0].set_ylabel("Energy")
axes[0].set_title("Energy components over time")
axes[0].grid(True)
axes[0].legend()

axes[1].plot(times_energy, relative_energy_change)
axes[1].axhline(0.0, color="black", linewidth=1)

```



```

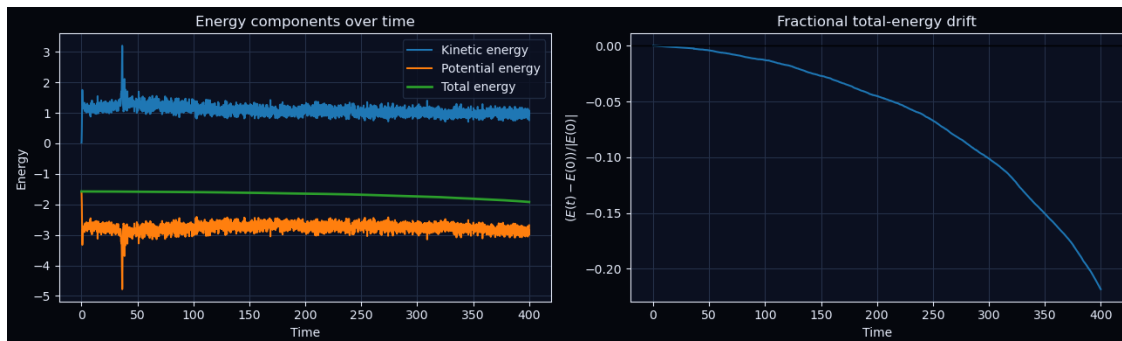
axes[1].set_xlabel("Time")
axes[1].set_ylabel(r" $(E(t) - E(0)) / |E(0)|$ ")
axes[1].set_title("Fractional total-energy drift")
axes[1].grid(True)

plt.tight_layout()
plt.show()

# Interpretation for the report:
#
# For an exactly isolated gravitational system, total energy should be
# conserved.
# Numerically, it will not be perfectly constant because of timestep error,
# force softening, and the chosen force approximation. RK4 is high-order
# accurate
# per timestep, but it is not a symplectic integrator, so over long integrations
# it can show secular energy drift. Symplectic schemes such as Leapfrog or
# Velocity-Verlet often conserve the long-term structure of the energy better
# in gravitational N-body problems, even though they are formally lower order.

```

Energy run runtime: 221.247 s
 Initial total energy: -1.577636
 Final total energy: -1.922286
 Final fractional energy drift: -2.185e-01
 Maximum absolute fractional drift: 2.185e-01



0.2.1 Breakdown of code cell above

This cell is organized into the following pieces:

- Imports the main numerical, plotting, statistics, and astronomy libraries used later in the notebook: `the`.
- Defines reusable functions used by later cells:
 - `total_energy_arrays` computes kinetic, potential, and total energy from array state.
 - `total_energy_particles` computes energy directly from particle objects.

- `evolve_fixed_timestep_selectable_with_energy` runs the optimized RK4 solver while recording energy.
- Sets up two Plummer-like galaxies separated in space and gives them bulk velocities toward one another.
- Evolves the system with the optimized RK4 solver while saving energy diagnostics for conservation checks.
- Builds Matplotlib figures for visual comparison or diagnostic plots.
- Computes and plots total-energy drift to assess numerical conservation.
- Measures runtime so different numerical methods can be compared quantitatively.

Over longer integration runs the energy drift becomes more apparent, when only around 80 seconds the right hand plot still shows a drift but its hard to see it visually in the left plot. Increasing the time to around 400 and you can start to see the drift visually in the left hand plot.

[]:

0.3 Differential rotational velocity initial conditions

The previous simulations used mainly bulk velocities, so the galaxies moved almost directly into each other. The next cell gives each galaxy internal differential rotation, where each particle receives a tangential velocity based on the mass enclosed inside its radius. A small impact parameter and tangential bulk velocity are also included to give the galaxy-galaxy encounter angular momentum.

```
[22]: # Differential rotational velocity initial conditions

def add_differential_rotation(
    particles,
    center=np.array([0.0, 0.0]),
    G=1.0,
    eps=0.05,
    rotation_scale=0.4,
    spin=1
):
    """
    Add differential rotational velocity to one galaxy.

    For each particle at radius  $r$  from the galaxy centre, use the project
    prescription

        
$$v(r) = \sqrt{G M(<r) / r}$$


    where  $M(<r)$  is the mass enclosed inside that particle's radius.

    The velocity is applied in the tangential direction:

        
$$e_{\phi} = (-y/r, x/r)$$

```

Parameters

particles : list

Particles belonging to one galaxy only.

center : ndarray

Centre of that galaxy.

rotation_scale : float

Multiplies the circular velocity. Values below 1 make the galaxy less rotationally supported and usually more numerically stable.

spin : int

+1 gives counter-clockwise rotation, -1 gives clockwise rotation.

"""

```
center = np.asarray(center, dtype=float)
```

```
positions = np.array([p.position for p in particles])
```

```
masses = np.array([p.mass for p in particles])
```

```
relative_positions = positions - center
```

```
radii = np.linalg.norm(relative_positions, axis=1)
```

```
for i, particle in enumerate(particles):
```

```
    r = radii[i]
```

```
    # Avoid dividing by zero for particles extremely close to the centre
```

```
    if r < eps:
```

```
        continue
```

```
    # Enclosed mass: sum of particle masses at radii smaller than this one
```

```
    enclosed_mass = np.sum(masses[radii <= r])
```

```
    # Differential circular speed from the project handout
```

```
    v_circular = np.sqrt(G * enclosed_mass / r)
```

```
    # Tangential unit vector around the galaxy centre
```

```
    x_rel, y_rel = relative_positions[i]
```

```
    tangential_direction = np.array([-y_rel / r, x_rel / r])
```

```
    particle.velocity += (
```

```
        spin
```

```
        * rotation_scale
```

```
        * v_circular
```

```
        * tangential_direction
```

```
)
```

```

def initialize_two_galaxies_differential_rotation(
    N_per_galaxy=50,
    d=8.0,
    impact_parameter=2.0,
    theta=0.5,
    radial_velocity=0.10,
    tangential_velocity=0.05,
    rotation_scale=0.4,
    spin_1=1,
    spin_2=-1,
    G=1.0,
    eps=0.05
):
    """
    Initialize two Plummer galaxies with differential internal rotation.

    Compared with initialize_two_galaxies, this function adds two new ideas:

    1. Each galaxy rotates internally using  $v(r) = \sqrt{G M(<r) / r}$ .
    2. The galaxy centres are given a small impact parameter and tangential
       velocity, so the encounter has angular momentum instead of being purely
       head-on.
    """

    galaxy_1 = initialize_particles(N_per_galaxy)
    galaxy_2 = initialize_particles(N_per_galaxy)

    # Place the galaxies with an offset in both x and y
    # The y-offset is the impact parameter.
    center_1 = np.array([-d / 2.0, -impact_parameter / 2.0])
    center_2 = np.array([d / 2.0, impact_parameter / 2.0])

    for particle in galaxy_1:
        particle.position += center_1

    for particle in galaxy_2:
        particle.position += center_2

    # Add internal differential rotation around each galaxy's own centre
    add_differential_rotation(
        galaxy_1,
        center=center_1,
        G=G,
        eps=eps,
        rotation_scale=rotation_scale,

```

```

        spin=spin_1
    )

    add_differential_rotation(
        galaxy_2,
        center=center_2,
        G=G,
        eps=eps,
        rotation_scale=rotation_scale,
        spin=spin_2
    )

    # Add bulk motion so the two galaxies approach each other
    # The y-components add orbital angular momentum to the encounter
    bulk_velocity_1 = np.array([ radial_velocity,  tangential_velocity])
    bulk_velocity_2 = np.array([-radial_velocity, -tangential_velocity])

    for particle in galaxy_1:
        particle.velocity += bulk_velocity_1

    for particle in galaxy_2:
        particle.velocity += bulk_velocity_2

    particles = galaxy_1 + galaxy_2

    return particles, theta

# Run the optimised simulation with differential rotation

# Set the random seed for reproducible initial conditions
np.random.seed(31)

particles_diffrot, theta_diffrot = ↳ initialize_two_galaxies_differential_rotation(
    N_per_galaxy=150,
    d=15.0,
    impact_parameter=2.0,
    theta=0.5,
    radial_velocity=0.10,
    tangential_velocity=0.05,
    rotation_scale=0.4,
    spin_1=1,
    spin_2=-1,
    G=1.0,

```

```

    eps=0.05
)

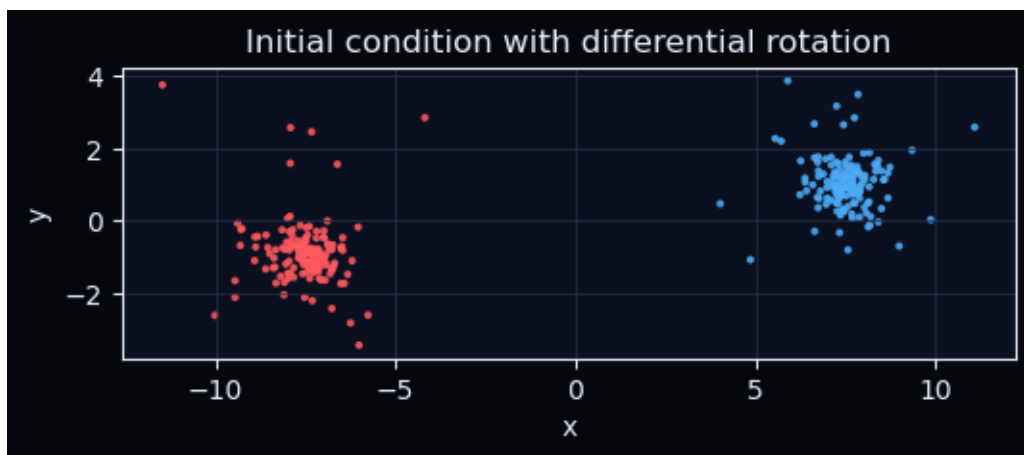
plot_particles(
    particles_diffrot,
    title="Initial condition with differential rotation",
    s=8
)

start = time.perf_counter()
times_diffrot, history_diffrot = evolve_fixed_timestep_selectable(
    particles_diffrot,
    t_end=80.0,
    dt=0.01,
    theta=theta_diffrot,
    G=1.0,
    eps=0.05,
    save_every=10,
    method="auto",
    direct_threshold=600
)
runtime_diffrot = time.perf_counter() - start

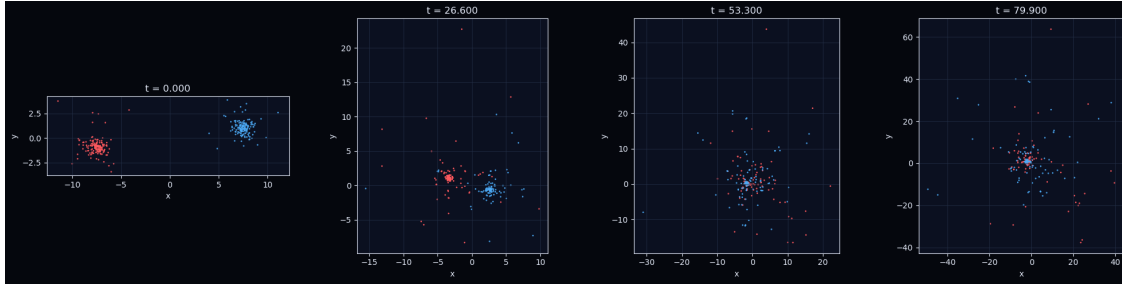
print(f"Differential-rotation run runtime: {runtime_diffrot:.3f} s")

plot_simulation_snapshots(
    history_diffrot,
    times=times_diffrot,
    s=4
)

```



Differential-rotation run runtime: 97.244 s



0.3.1 Breakdown of code cell above

This cell is organized into the following pieces:

- Defines reusable functions used by later cells:
 - `add_differential_rotation` adds tangential velocities based on enclosed mass.
 - `initialize_two_galaxies_differential_rotation` builds a rotating two-galaxy encounter with impact parameter.
- Sets up a two-galaxy encounter with internal differential rotation, an impact parameter, and bulk orbital motion.
- Evolves the system with the optimized selectable force calculation, using direct summation for small particle numbers.
- Plots stored snapshots so the merger morphology can be compared at several times.
- Measures runtime so different numerical methods can be compared quantitatively.

```
[23]: # Animate the differential-rotation simulation
animate_simulation(
    history_diffrot,
    times=times_diffrot,
    interval=80,
    s=4,
    zoom=10
)
```

Animation size has reached 21002893 bytes, exceeding the limit of 20971520.0. If you're sure you want a larger animation embedded, set the `animation.embed_limit` rc parameter to a larger value (in MB). This and further frames will be dropped.

```
[23]: <IPython.core.display.HTML object>
```

```
[ ]:
```

0.4 Euler versus RK4 integration

The next cell compares the original RK4 integrator with a simple forward Euler method. Both simulations start from identical differential-rotation initial conditions, so differences in the trajectories and energy drift are due to the integration method rather than different random initial particle positions.

```
[24]: # Euler integrator comparison against RK4
```

```
def euler_step_selectable(
    particles,
    dt,
    theta=0.5,
    G=1.0,
    eps=0.05,
    method="auto",
    direct_threshold=600
):
    """
    Advance the system by one forward Euler timestep.

    Euler is the simplest possible integrator:

         $x(t + dt) = x(t) + v(t) dt$ 
         $v(t + dt) = v(t) + a(t) dt$ 

    It only evaluates the acceleration once per step, so it is fast, but it is
    only first-order accurate and usually performs poorly for long gravitational
    simulations.
    """

    positions, velocities, masses = get_state(particles)

    use_direct = (
        method == "direct"
        or (method == "auto" and len(masses) <= direct_threshold)
    )

    if use_direct:
        accelerations = compute_accelerations_direct_arrays(
            positions,
            masses,
            G=G,
            eps=eps
        )
    else:
        accelerations = compute_accelerations_barnes_hut(
            particles,
            theta=theta,
            G=G,
            eps=eps
        )
```



```

new_positions = positions + dt * velocities
new_velocities = velocities + dt * accelerations

set_state(particles, new_positions, new_velocities)

def evolve_euler_with_energy(
    particles,
    t_end,
    dt,
    theta=0.5,
    G=1.0,
    eps=0.05,
    save_every=1,
    method="auto",
    direct_threshold=600
):
    """
    Evolve the system using forward Euler while recording positions and energy.
    """

    times = []
    position_history = []
    total_energy_history = []
    kinetic_history = []
    potential_history = []

    t = 0.0
    step = 0

    def save_current_state(current_time):
        positions, velocities, masses = get_state(particles)
        total_E, kinetic_E, potential_E = total_energy_arrays(
            positions,
            velocities,
            masses,
            G=G,
            eps=eps
        )

        times.append(current_time)
        position_history.append(positions.copy())
        total_energy_history.append(total_E)
        kinetic_history.append(kinetic_E)
        potential_history.append(potential_E)

    while t < t_end:

```

```

    if step % save_every == 0:
        save_current_state(t)

    current_dt = min(dt, t_end - t)

    euler_step_selectable(
        particles,
        current_dt,
        theta=theta,
        G=G,
        eps=eps,
        method=method,
        direct_threshold=direct_threshold
    )

    t += current_dt
    step += 1

    if len(times) == 0 or not np.isclose(times[-1], t_end):
        save_current_state(t_end)

    return (
        np.array(times),
        np.array(position_history),
        np.array(total_energy_history),
        np.array(kinetic_history),
        np.array(potential_history)
    )

# Run Euler and RK4 from exactly the same initial conditions

# These parameters match the gentler differential-rotation setup
# Use the same values as the differential-rotation setup for the cleanest
↪ comparison
# Set the random seed for reproducible initial conditions
np.random.seed(41)

initial_particles_integrator_compare, theta_integrator_compare = (
    initialize_two_galaxies_differential_rotation(
        N_per_galaxy=150,
        d=15.0,
        impact_parameter=2,
        theta=0.5,
        radial_velocity=0.10,

```

```

        tangential_velocity=0.05,
        rotation_scale=0.4,
        spin_1=1,
        spin_2=-1,
        G=1.0,
        eps=0.05
    )
)

particles_rk4_compare = clone_particles(initial_particles_integrator_compare)
particles_euler_compare = clone_particles(initial_particles_integrator_compare)

compare_t_end = 80.0
compare_dt = 0.01
compare_eps = 0.05
compare_save_every = 10

start = time.perf_counter()
(
    times_rk4_compare,
    history_rk4_compare,
    velocity_history_rk4_compare,
    total_energy_rk4_compare,
    kinetic_energy_rk4_compare,
    potential_energy_rk4_compare
) = evolve_fixed_timestep_selectable_with_energy(
    particles_rk4_compare,
    t_end=compare_t_end,
    dt=compare_dt,
    theta=theta_integrator_compare,
    G=1.0,
    eps=compare_eps,
    save_every=compare_save_every,
    method="auto",
    direct_threshold=600
)
runtime_rk4_compare = time.perf_counter() - start

start = time.perf_counter()
(
    times_euler_compare,
    history_euler_compare,
    total_energy_euler_compare,
    kinetic_energy_euler_compare,
    potential_energy_euler_compare
) = evolve_euler_with_energy(
    particles_euler_compare,

```

```

        t_end=compare_t_end,
        dt=compare_dt,
        theta=theta_integrator_compare,
        G=1.0,
        eps=compare_eps,
        save_every=compare_save_every,
        method="auto",
        direct_threshold=600
    )
runtime_euler_compare = time.perf_counter() - start

# Print energy drift summaries

relative_energy_rk4 = (
    total_energy_rk4_compare - total_energy_rk4_compare[0]
) / abs(total_energy_rk4_compare[0])

relative_energy_euler = (
    total_energy_euler_compare - total_energy_euler_compare[0]
) / abs(total_energy_euler_compare[0])

print(f"RK4 runtime:    {runtime_rk4_compare:.3f} s")
print(f"Euler runtime: {runtime_euler_compare:.3f} s")
print()
print(f"RK4 final fractional energy drift:    {relative_energy_rk4[-1]:.3e}")
print(f"Euler final fractional energy drift: {relative_energy_euler[-1]:.3e}")
print()
print(f"RK4 maximum absolute drift:    {np.max(np.abs(relative_energy_rk4)):.3e}")
print(f"Euler maximum absolute drift: {np.max(np.abs(relative_energy_euler)):.3e}")

# Plot the final states and energy drift

# Create or configure the Matplotlib figure
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
fig.patch.set_facecolor(DARK_BACKGROUND)

scatter_galaxy_positions(axes[0, 0], history_rk4_compare[-1], s=4)
axes[0, 0].set_title("RK4 final state")
axes[0, 0].set_aspect("equal")

```

```

style_dark_axis(axes[0, 0])

scatter_galaxy_positions(axes[0, 1], history_euler_compare[-1], s=4)
axes[0, 1].set_title("Euler final state")
axes[0, 1].set_aspect("equal")
style_dark_axis(axes[0, 1])

axes[1, 0].plot(times_rk4_compare, relative_energy_rk4, label="RK4")
axes[1, 0].plot(times_euler_compare, relative_energy_euler, label="Euler")
axes[1, 0].axhline(0.0, color="black", linewidth=1)
axes[1, 0].set_xlabel("Time")
axes[1, 0].set_ylabel(r" $(E(t) - E(0)) / |E(0)|$ ")
axes[1, 0].set_title("Total-energy drift")
style_dark_axis(axes[1, 0])
axes[1, 0].legend()

axes[1, 1].plot(times_rk4_compare, total_energy_rk4_compare, label="RK4")
axes[1, 1].plot(times_euler_compare, total_energy_euler_compare, label="Euler")
axes[1, 1].set_xlabel("Time")
axes[1, 1].set_ylabel("Total energy")
axes[1, 1].set_title("Total energy over time")
style_dark_axis(axes[1, 1])
axes[1, 1].legend()

for ax in axes[0, :]:
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    style_dark_axis(ax)

plt.tight_layout()
plt.show()

```

```

# Interpretation for the report:
#
# Euler is faster because it only computes the force once per timestep, while
# RK4 computes it four times. However, Euler is much less accurate. In an
# N-body gravitational system this usually appears as much larger energy drift
# and less reliable trajectories. RK4 should therefore produce a more stable
# and physically trustworthy merger, even though it is more expensive.

```

RK4 runtime: 99.819 s

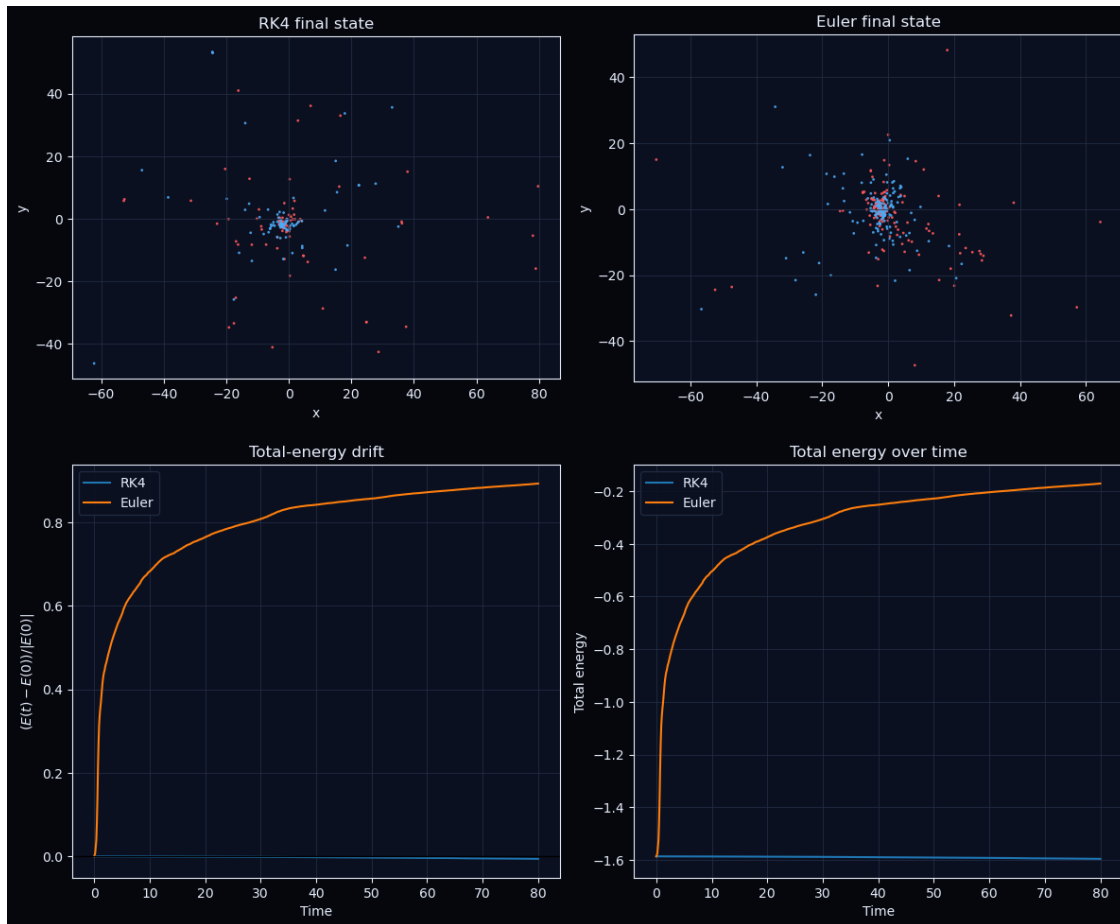
Euler runtime: 27.297 s

RK4 final fractional energy drift: -5.961e-03

Euler final fractional energy drift: 8.924e-01

RK4 maximum absolute drift: 5.961e-03

Euler maximum absolute drift: 8.924e-01



0.4.1 Breakdown of code cell above

This cell is organized into the following pieces:

- Defines reusable functions used by later cells:
 - `euler_step_selectable` advances the system by one forward Euler step.
 - `evolve_euler_with_energy` runs Euler integration while recording energy diagnostics.
- Sets up a two-galaxy encounter with internal differential rotation, an impact parameter, and bulk orbital motion.
- Evolves the system with the optimized RK4 solver while saving energy diagnostics for conservation checks.
- Builds Matplotlib figures for visual comparison or diagnostic plots.
- Computes and plots total-energy drift to assess numerical conservation.
- Measures runtime so different numerical methods can be compared quantitatively.

```
[25]: # Animate the Euler simulation to see how the poorer integrator behaves, ␣
      ↪ spoiler its very bad ahhahahaha
```

```

animate_simulation(
    history_euler_compare,
    times=times_euler_compare,
    interval=80,
    s=4,
    zoom = 10
)

```

Animation size has reached 21019266 bytes, exceeding the limit of 20971520.0. If you're sure you want a larger animation embedded, set the `animation.embed_limit` rc parameter to a larger value (in MB). This and further frames will be dropped.

[25]: <IPython.core.display.HTML object>

[]:

0.5 Exploring an interesting differential-rotation merger

This section investigates a more visually interesting RK4 simulation by increasing the angular momentum of the galaxy-galaxy encounter. The main changes are a larger impact parameter and a larger tangential bulk velocity, which should produce a grazing merger with curved trajectories, tidal distortions, and possibly bridge or tail-like structures.

```

[26]: # Interesting-result search: grazing differential-rotation merger

# The previous differential-rotation run was already more realistic than the
# head-on case, but this section intentionally increases the orbital angular
# momentum of the encounter. A larger impact parameter and tangential velocity
# should make the galaxies curve around each other before merging.

# Three short candidate runs are used to compare encounter geometries.
interesting_candidates = [
    {
        "label": "mild grazing",
        "impact_parameter": 2.5,
        "tangential_velocity": 0.06,
        "radial_velocity": 0.08,
        "rotation_scale": 0.40,
    },
    {
        "label": "strong grazing",
        "impact_parameter": 4.0,
        "tangential_velocity": 0.10,
        "radial_velocity": 0.07,
        "rotation_scale": 0.45,
    },
    {

```

```

        "label": "wide fly-by test",
        "impact_parameter": 5.0,
        "tangential_velocity": 0.12,
        "radial_velocity": 0.06,
        "rotation_scale": 0.45,
    },
]

candidate_results = []

for candidate_index, candidate in enumerate(interesting_candidates):
    # Each candidate receives its own reproducible random seed.
    np.random.seed(100 + candidate_index)

    particles_candidate, theta_candidate = □
    ↪ initialize_two_galaxies_differential_rotation(
        N_per_galaxy=60,
        d=10.0,
        impact_parameter=candidate["impact_parameter"],
        theta=0.5,
        radial_velocity=candidate["radial_velocity"],
        tangential_velocity=candidate["tangential_velocity"],
        rotation_scale=candidate["rotation_scale"],
        spin_1=1,
        spin_2=-1,
        G=1.0,
        eps=0.07
    )

    start = time.perf_counter()
    times_candidate, history_candidate = evolve_fixed_timestep_selectable(
        particles_candidate,
        t_end=22.0,
        dt=0.02,
        theta=theta_candidate,
        G=1.0,
        eps=0.07,
        save_every=10,
        method="auto",
        direct_threshold=300
    )
    runtime_candidate = time.perf_counter() - start

    candidate_results.append(
        {
            "parameters": candidate,
            "times": times_candidate,

```



```

        "history": history_candidate,
        "runtime": runtime_candidate,
    }
)

print(
    f"{candidate['label']} completed in {runtime_candidate:.2f} s "
    f"with impact_parameter={candidate['impact_parameter']} and "
    f"tangential_velocity={candidate['tangential_velocity']}"
)

# Compare the final state of each candidate
fig, axes = plt.subplots(1, len(candidate_results), figsize=(15, 4))
fig.patch.set_facecolor(DARK_BACKGROUND)

for ax, result in zip(axes, candidate_results):
    final_positions = result["history"][-1]
    label = result["parameters"]["label"]

    scatter_galaxy_positions(ax, final_positions, s=4)
    ax.set_title(label)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_aspect("equal")
    style_dark_axis(ax)

plt.tight_layout()
plt.show()

#Lets now chose one of these potentiall interesting cases, I want to vibe check
↪ a fly by case
np.random.seed(101)

particles_interesting, theta_interesting =
    ↪ initialize_two_galaxies_differential_rotation(
        N_per_galaxy=100,
        d=10.0,
        impact_parameter=12.0,
        theta=0.5,
        radial_velocity=0.225,
        tangential_velocity=0.04,
        rotation_scale=0.4,
        spin_1=1,
        spin_2=-1,
        G=1.0,
    )

```

```

    eps=0.05
)

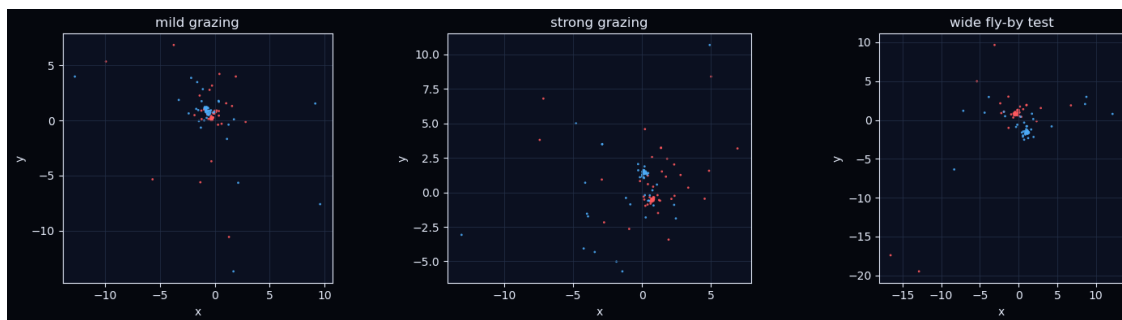
start = time.perf_counter()
times_interesting, history_interesting = evolve_fixed_timestep_selectable(
    particles_interesting,
    t_end=110.0,
    dt=0.01,
    theta=theta_interesting,
    G=1.0,
    eps=0.05,
    save_every=15,
    method="auto",
    direct_threshold=600
)
runtime_interesting = time.perf_counter() - start

print(f"Interesting showcase run completed in {runtime_interesting:.2f} s")

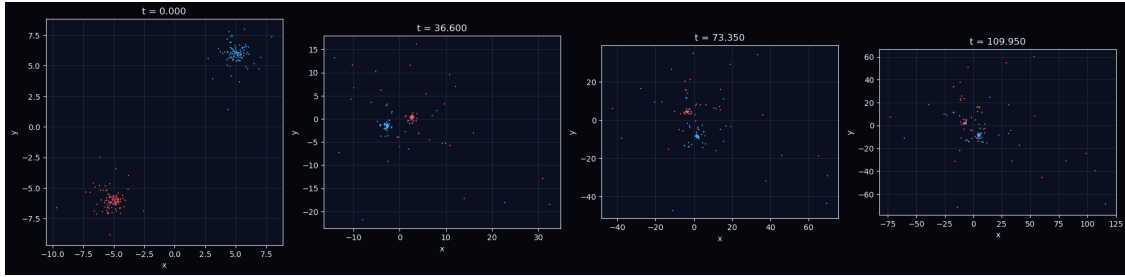
plot_simulation_snapshots(
    history_interesting,
    times=times_interesting,
    s=3
)

```

mild grazing completed in 2.28 s with impact_parameter=2.5 and
 tangential_velocity=0.06
 strong grazing completed in 2.25 s with impact_parameter=4.0 and
 tangential_velocity=0.1
 wide fly-by test completed in 2.18 s with impact_parameter=5.0 and
 tangential_velocity=0.12



Interesting showcase run completed in 60.91 s



```
[27]: # Animate the selected interesting differential-rotation merger
animate_simulation(
    history_interesting,
    times=times_interesting,
    interval=80,
    s=3,
    zoom = 10
)
```

Animation size has reached 20971938 bytes, exceeding the limit of 20971520.0. If you're sure you want a larger animation embedded, set the animation.embed_limit rc parameter to a larger value (in MB). This and further frames will be dropped.

```
[27]: <IPython.core.display.HTML object>
```

```
[28]: # Parameter playground for interesting differential-rotation cases

# I should only have to edit this section when testing new cases
playground_params = {
    "label": "Best Merger?",

    # Random seed controls the exact particle realization
    # Change this to test a different random galaxy with the same parameters
    "seed": 123,

    # Galaxy setup
    "N_per_galaxy": 150,
    "d": 10.0,
    "impact_parameter": 12.0,

    # Velocity setup
    "radial_velocity": 0.1,
    "tangential_velocity": 0.22,
    "rotation_scale": 0.3,
    "spin_1": 1,
    "spin_2": -1,
```

```

# Numerical settings
"theta": 0.5,
"eps": 0.05,
"t_end": 90.0,
"dt": 0.01,
"save_every": 15,
"direct_threshold": 600,

# Plot settings
"scatter_size": 3,
"make_animation": True,
}

def run_playground_merger(params):
    """
    Run one custom differential-rotation merger without modifying earlier
    ↪ results.

    This function creates fresh particles every time, runs the optimized RK4
    solver, plots snapshots, and optionally returns an animation.
    """

    np.random.seed(params["seed"])

    playground_particles, playground_theta =
    ↪ initialize_two_galaxies_differential_rotation(
        N_per_galaxy=params["N_per_galaxy"],
        d=params["d"],
        impact_parameter=params["impact_parameter"],
        theta=params["theta"],
        radial_velocity=params["radial_velocity"],
        tangential_velocity=params["tangential_velocity"],
        rotation_scale=params["rotation_scale"],
        spin_1=params["spin_1"],
        spin_2=params["spin_2"],
        G=1.0,
        eps=params["eps"]
    )

    print("Running playground case:")
    print(f"  label = {params['label']}")
    print(f"  N_total = {2 * params['N_per_galaxy']}")
    print(f"  impact_parameter = {params['impact_parameter']}")
    print(f"  tangential_velocity = {params['tangential_velocity']}")
    print(f"  rotation_scale = {params['rotation_scale']}")

```

```

start = time.perf_counter()

playground_times, playground_history = evolve_fixed_timestep_selectable(
    playground_particles,
    t_end=params["t_end"],
    dt=params["dt"],
    theta=playground_theta,
    G=1.0,
    eps=params["eps"],
    save_every=params["save_every"],
    method="auto",
    direct_threshold=params["direct_threshold"]
)

playground_runtime = time.perf_counter() - start

print(f"Runtime: {playground_runtime:.2f} s")

plot_simulation_snapshots(
    playground_history,
    times=playground_times,
    s=params["scatter_size"]
)

result = {
    "params": params.copy(),
    "particles": playground_particles,
    "times": playground_times,
    "history": playground_history,
    "runtime": playground_runtime,
}

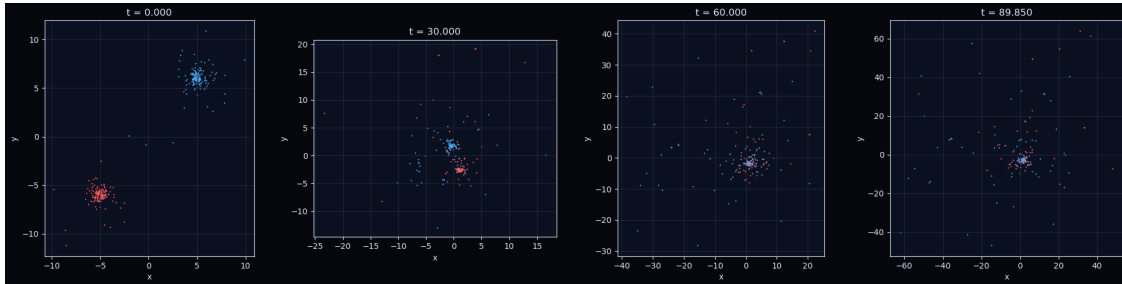
if params["make_animation"]:
    result["animation"] = animate_simulation(
        playground_history,
        times=playground_times,
        interval=80,
        s=params["scatter_size"]
    )

return result

playground_result = run_playground_merger(playground_params)

```

```
Running playground case:
label = Best Merger?
N_total = 300
impact_parameter = 12.0
tangential_velocity = 0.22
rotation_scale = 0.3
Runtime: 111.38 s
```



```
[29]: animate_simulation(
    playground_result["history"],
    times=playground_result["times"],
    interval=80,
    s=3,
    zoom=10
)
```

Animation size has reached 20975662 bytes, exceeding the limit of 20971520.0. If you're sure you want a larger animation embedded, set the `animation.embed_limit` rc parameter to a larger value (in MB). This and further frames will be dropped.

```
[29]: <IPython.core.display.HTML object>
```

```
[ ]:
```

0.6 Multiple playground simulations

This section runs several custom differential-rotation playground cases in sequence. Each case changes only a few initial-condition parameters, while the rest are inherited from `playground_params`. The results are stored in `playground_multi_results`, so individual cases can be plotted or animated after the batch has finished.

```
[30]: # Run multiple playground simulations with different parameters

# This cell assumes that playground_params and run_playground_merger have
↳ already
# been defined in the previous playground section
```

```

# Turn off automatic animation during the batch run so the notebook stays fast
# and does not create several large HTML animations at once. This was suggested
↳by Gen-AI when I ran into a runtime issue
base_playground_params = playground_params.copy()
base_playground_params["make_animation"] = False

# Each case below starts from the same base settings and then changes the
# parameters that define a particular encounter geometry.
playground_cases = [
    {
        **base_playground_params,
        "label": "head-on low values",
        "seed": 201,
        "d": 10.0,
        "impact_parameter": 0.5,
        "radial_velocity": 0.05,
        "tangential_velocity": 0.01,
        "rotation_scale": 0.1,
    },
    {
        **base_playground_params,
        "label": "grazing merger",
        "seed": 202,
        "d": 10.0,
        "impact_parameter": 20.0,
        "radial_velocity": 0.1,
        "tangential_velocity": 0.08,
        "rotation_scale": 0.4,
    },
    {
        **base_playground_params,
        "label": "Fly Through",
        "seed": 203,
        "d": 15.0,
        "impact_parameter": 30.0,
        "radial_velocity": 1.0,
        "tangential_velocity": 2.0,
        "rotation_scale": 1.0,
    },
]

# Store all outputs here so the simulations can be compared or animated later
playground_multi_results = []

for case in playground_cases:
    print("=" * 70)

```

```

print(f"Running case: {case['label']}")
print("=" * 70)
print(f"impact_parameter    = {case['impact_parameter']}")
print(f"radial_velocity      = {case['radial_velocity']}")
print(f"tangential_velocity= {case['tangential_velocity']}")
print(f"rotation_scale       = {case['rotation_scale']}")

result = run_playground_merger(case)
playground_multi_results.append(result)

# Compact comparison of the final state from each run

fig, axes = plt.subplots(
    1,
    len(playground_multi_results),
    figsize=(5 * len(playground_multi_results), 5)
)
fig.patch.set_facecolor(DARK_BACKGROUND)

if len(playground_multi_results) == 1:
    axes = [axes]

for ax, result in zip(axes, playground_multi_results):
    final_positions = result["history"][-1]
    label = result["params"]["label"]

    scatter_galaxy_positions(ax, final_positions, s=3)
    ax.set_title(label)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_aspect("equal")
    style_dark_axis(ax)

plt.tight_layout()
plt.show()

```

```

=====
Running case: head-on low values
=====

```

```

impact_parameter    = 0.5
radial_velocity     = 0.05
tangential_velocity = 0.01
rotation_scale      = 0.1
Running playground case:
    label = head-on low values
    N_total = 300

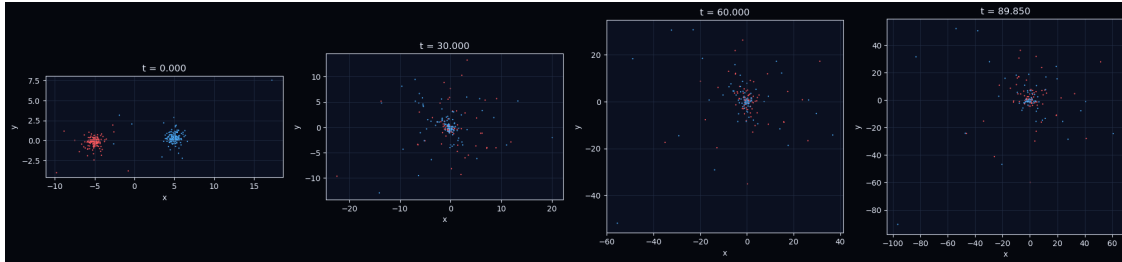
```



```

impact_parameter = 0.5
tangential_velocity = 0.01
rotation_scale = 0.1
Runtime: 110.67 s

```



```

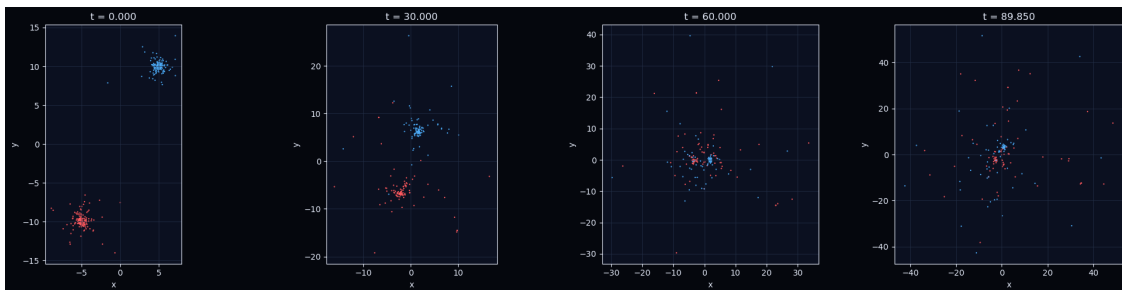
=====
Running case: grazing merger
=====

```

```

impact_parameter    = 20.0
radial_velocity     = 0.1
tangential_velocity = 0.08
rotation_scale      = 0.4
Running playground case:
  label = grazing merger
  N_total = 300
  impact_parameter = 20.0
  tangential_velocity = 0.08
  rotation_scale = 0.4
Runtime: 110.95 s

```



```

=====
Running case: Fly Through
=====

```

```

impact_parameter    = 30.0
radial_velocity     = 1.0
tangential_velocity = 2.0
rotation_scale      = 1.0

```

Running playground case:

label = Fly Through

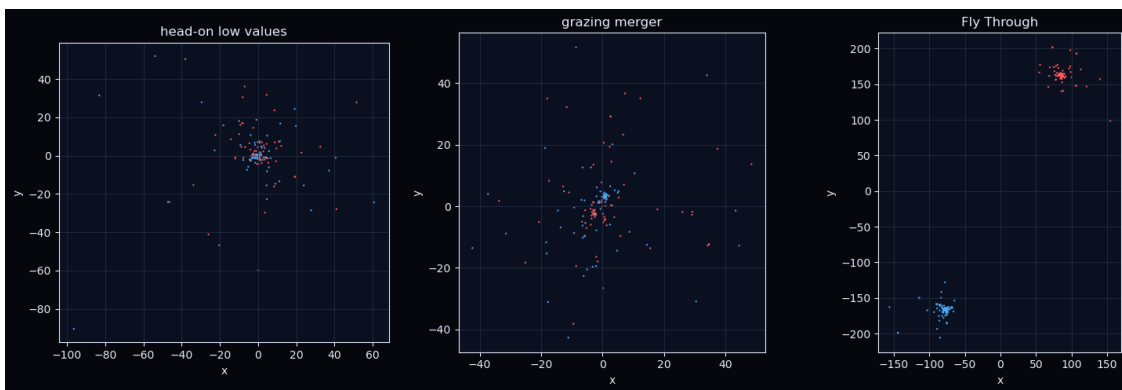
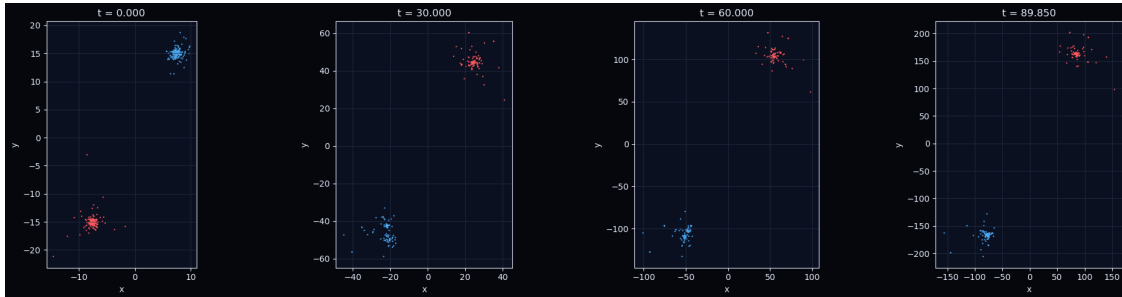
N_total = 300

impact_parameter = 30.0

tangential_velocity = 2.0

rotation_scale = 1.0

Runtime: 110.50 s



```
[31]: # Choose which case to animate:
#      0 = head-on low angular momentum
#      1 = grazing merger
#      2 = fly through
case_index = 0

selected_case = playground_multi_results[case_index]

print(f"Animating case: {selected_case['params']['label']}")

animate_simulation(
    selected_case["history"],
    times=selected_case["times"],
    interval=80,
```

```
s=selected_case["params"]["scatter_size"],
zoom=10
)
```

Animating case: head-on low values

Animation size has reached 20985842 bytes, exceeding the limit of 20971520.0. If you're sure you want a larger animation embedded, set the animation.embed_limit rc parameter to a larger value (in MB). This and further frames will be dropped.

[31]: <IPython.core.display.HTML object>

```
[32]: # Choose which case to animate:
#      0 = head-on low angular momentum
#      1 = grazing merger
#      2 = wide fly-by
case_index = 1

selected_case = playground_multi_results[case_index]

print(f"Animating case: {selected_case['params']['label']}")

animate_simulation(
    selected_case["history"],
    times=selected_case["times"],
    interval=80,
    s=selected_case["params"]["scatter_size"],
    zoom=10
)
```

Animating case: grazing merger

Animation size has reached 20981596 bytes, exceeding the limit of 20971520.0. If you're sure you want a larger animation embedded, set the animation.embed_limit rc parameter to a larger value (in MB). This and further frames will be dropped.

[32]: <IPython.core.display.HTML object>

```
[33]: # Choose which case to animate:
#      0 = head-on low angular momentum
#      1 = grazing merger
#      2 = Fly Through
case_index = 2

selected_case = playground_multi_results[case_index]

print(f"Animating case: {selected_case['params']['label']}")

animate_simulation(
```

```

selected_case["history"],
times=selected_case["times"],
interval=80,
s=selected_case["params"]["scatter_size"],
zoom=10
)

```

Animating case: Fly Through

[33]: <IPython.core.display.HTML object>

[]:

[]:

0.7 Exporting figures and animations for the report

0.7.1 This code was written in bulk by Gen-AI as it helped me just collate my results and export them in a format I can use for my submission that is all.

This final section saves the main simulation outputs into a separate `results/` folder.

```

[34]: # =====
# Export report figures and animations
# =====

from pathlib import Path
import json
from datetime import datetime
from matplotlib import animation as mpl_animation

# Store exports beside the notebook in a clearly named results folder.
PROJECT_DIR = Path(
    "/Users/williamwall/Documents/Amsterdam/UvA/Period5/Computational/
    ↪Comp_Project"
)
RESULTS_DIR = PROJECT_DIR / "results"
FIGURES_DIR = RESULTS_DIR / "figures"
ANIMATIONS_DIR = RESULTS_DIR / "animations"
DATA_DIR = RESULTS_DIR / "data"

for directory in [RESULTS_DIR, FIGURES_DIR, ANIMATIONS_DIR, DATA_DIR]:
    directory.mkdir(parents=True, exist_ok=True)

EXPORT_MANIFEST = {
    "created": datetime.now().isoformat(timespec="seconds"),

```

```

    "figures": [],
    "animations": [],
    "data": [],
    "notes": [],
}

def _json_safe(value):
    """
    Convert NumPy values and arrays into JSON-safe Python objects.
    """

    if isinstance(value, np.ndarray):
        return value.tolist()

    if isinstance(value, (np.integer, np.floating)):
        return value.item()

    if isinstance(value, dict):
        return {str(key): _json_safe(item) for key, item in value.items()}

    if isinstance(value, (list, tuple)):
        return [_json_safe(item) for item in value]

    return value

def _safe_name(name):
    """
    Convert a display name into a clean filename stem.
    """

    safe = str(name).strip().lower()
    safe = safe.replace(" ", "_")
    safe = "".join(char for char in safe if char.isalnum() or char in "_-")
    return safe or "unnamed_output"

def _thin_history(history, times=None, max_frames=None):
    """
    Optionally downsample simulation histories before animation export.

    If max_frames is None, the full saved history is used. This is the
    preferred setting for full-length report and presentation videos.
    """

    history = np.asarray(history)

```

```

    if times is not None:
        times = np.asarray(times)

    n_frames = len(history)

    if max_frames is None or n_frames <= max_frames:
        return history, times

    indices = np.linspace(0, n_frames - 1, max_frames).astype(int)
    indices = np.unique(indices)

    thinned_history = history[indices]

    if times is None:
        thinned_times = None
    else:
        thinned_times = times[indices]

    return thinned_history, thinned_times

def save_snapshot_grid(
    history,
    times=None,
    name="simulation",
    frames=None,
    s=3,
    zoom=None
):
    """
    Save a four-panel snapshot figure for one simulation history.
    """

    history = np.asarray(history)

    if frames is None:
        frames = [0, len(history)//3, 2*len(history)//3, len(history)-1]

    filename = FIGURES_DIR / f"{_safe_name(name)}_snapshots.png"

    fig, axes = plt.subplots(1, len(frames), figsize=(5 * len(frames), 5))
    fig.patch.set_facecolor(DARK_BACKGROUND)

    if len(frames) == 1:
        axes = [axes]

```

```

for ax, frame in zip(axes, frames):
    positions = history[frame]
    scatter_galaxy_positions(ax, positions, s=s)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_aspect("equal")

    if zoom is not None:
        ax.set_xlim(-zoom, zoom)
        ax.set_ylim(-zoom, zoom)

    if times is not None:
        ax.set_title(f"{name}, t = {times[frame]:.3f}")
    else:
        ax.set_title(f"{name}, frame = {frame}")

    style_dark_axis(ax)

plt.tight_layout()
fig.savefig(filename, dpi=220, bbox_inches="tight", facecolor=fig.
↪get_facecolor())
plt.close(fig)

EXPORT_MANIFEST["figures"].append(str(filename))
return filename

def save_single_frame(
    positions,
    name="final_state",
    title=None,
    s=3,
    zoom=None
):
    """
    Save a single scatter plot of particle positions.
    """

    filename = FIGURES_DIR / f"{_safe_name(name)}.png"

    fig, ax = plt.subplots(figsize=(6, 6))
    fig.patch.set_facecolor(DARK_BACKGROUND)
    scatter_galaxy_positions(ax, positions, s=s)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_aspect("equal")

```

```

if zoom is not None:
    ax.set_xlim(-zoom, zoom)
    ax.set_ylim(-zoom, zoom)

ax.set_title(title or name)
style_dark_axis(ax)

fig.savefig(filename, dpi=220, bbox_inches="tight", facecolor=fig.
↪get_facecolor())
plt.close(fig)

EXPORT_MANIFEST["figures"].append(str(filename))
return filename

def _make_history_animation(history, times=None, interval=80, s=3, zoom=None):
    """
    Create a Matplotlib animation object from a simulation history.
    """

    history = np.asarray(history)

    fig, ax = plt.subplots(figsize=(6, 6))
    fig.patch.set_facecolor(DARK_BACKGROUND)

    if zoom is None:
        all_x = history[:, :, 0]
        all_y = history[:, :, 1]
        x_min, x_max = np.min(all_x), np.max(all_x)
        y_min, y_max = np.min(all_y), np.max(all_y)
        padding = 0.1 * max(x_max - x_min, y_max - y_min)
        ax.set_xlim(x_min - padding, x_max + padding)
        ax.set_ylim(y_min - padding, y_max + padding)
    else:
        ax.set_xlim(-zoom, zoom)
        ax.set_ylim(-zoom, zoom)

    ax.set_aspect("equal")
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    style_dark_axis(ax)

    scatter = scatter_galaxy_positions(ax, history[0], s=s)

    def update(frame):
        positions = history[frame]
        scatter.set_offsets(positions)

```



```

        if times is not None:
            ax.set_title(f"t = {times[frame]:.3f}")
        else:
            ax.set_title(f"frame = {frame}")

    return scatter,

anim = mpl_animation.FuncAnimation(
    fig,
    update,
    frames=len(history),
    interval=interval,
    blit=True
)

return anim, fig

def save_animation_file(
    history,
    times=None,
    name="simulation",
    interval=80,
    s=3,
    zoom=None,
    max_frames=220,
    prefer="mp4"
):
    """
    Save one simulation history as a video-like file.

    The function saves MP4 when ffmpeg is available. If ffmpeg is unavailable,
    it falls back to HTML rather than GIF so the output remains suitable for
    reviewing the full animation in a browser.
    """

    history, times = _thin_history(history, times=times, max_frames=max_frames)
    anim, fig = _make_history_animation(
        history,
        times=times,
        interval=interval,
        s=s,
        zoom=zoom
    )

    stem = _safe_name(name)

```

```

saved_path = None

try:
    if prefer == "mp4" and mpl_animation.writers.is_available("ffmpeg"):
        saved_path = ANIMATIONS_DIR / f"{stem}.mp4"
        anim.save(
            saved_path,
            writer="ffmpeg",
            dpi=160,
            fps=max(1, int(1000 / interval))
        )

    else:
        saved_path = ANIMATIONS_DIR / f"{stem}.html"
        saved_path.write_text(anim.to_jshtml(), encoding="utf-8")
        EXPORT_MANIFEST["notes"].append(
            f"Animation {name} was saved as HTML because ffmpeg was not
↪available."
        )

    except Exception as error:
        saved_path = ANIMATIONS_DIR / f"{stem}.html"
        saved_path.write_text(anim.to_jshtml(), encoding="utf-8")
        EXPORT_MANIFEST["notes"].append(
            f"Animation {name} fell back to HTML because MP4 export failed:
↪{error}"
        )

plt.close(fig)

EXPORT_MANIFEST["animations"].append(str(saved_path))
return saved_path

def save_energy_diagnostic_plot(
    times,
    total_energy,
    kinetic_energy=None,
    potential_energy=None,
    name="energy_diagnostic"
):
    """
    Save an energy-conservation figure for one simulation.
    """

    times = np.asarray(times)
    total_energy = np.asarray(total_energy)

```

```

relative_energy = (total_energy - total_energy[0]) / abs(total_energy[0])

filename = FIGURES_DIR / f"{_safe_name(name)}.png"

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
fig.patch.set_facecolor(DARK_BACKGROUND)

if kinetic_energy is not None:
    axes[0].plot(times, kinetic_energy, label="Kinetic energy",
    ↪color=GALAXY_2_COLOR)

if potential_energy is not None:
    axes[0].plot(times, potential_energy, label="Potential energy",
    ↪color=GALAXY_1_COLOR)

axes[0].plot(times, total_energy, label="Total energy", color="#9be564",
    ↪linewidth=2)
axes[0].set_xlabel("Time")
axes[0].set_ylabel("Energy")
axes[0].set_title("Energy components")
axes[0].legend()
style_dark_axis(axes[0])

axes[1].plot(times, relative_energy, color="#f9c74f")
axes[1].axhline(0.0, color=DARK_TEXT, linewidth=1)
axes[1].set_xlabel("Time")
axes[1].set_ylabel(r" $(E(t)-E(0))/|E(0)|$ ")
axes[1].set_title("Fractional total-energy drift")
style_dark_axis(axes[1])

plt.tight_layout()
fig.savefig(filename, dpi=220, bbox_inches="tight", facecolor=fig.
    ↪get_facecolor())
plt.close(fig)

EXPORT_MANIFEST["figures"].append(str(filename))
return filename

def register_history_case(name, history_name, times_name=None, s=3, zoom=None):
    """
    Add a simulation to the export list if its history exists in memory.
    """

    if history_name not in globals():
        return None

```

```

history = globals()[history_name]
times = globals().get(times_name) if times_name is not None else None

return {
    "name": name,
    "history": history,
    "times": times,
    "s": s,
    "zoom": zoom,
}

def export_available_results(max_animation_frames=None, export_animations=True):
    """
    Export the main available notebook results.

    Run the notebook cells that generate the desired histories before running
    this function. Any missing histories are skipped automatically. By
    default, max_animation_frames=None saves full-length videos using every
    stored simulation frame.
    """

    cases = [
        register_history_case("fixed_timestep_rk4", "history_fixed", ↵
↵ "times_fixed", s=4),
        register_history_case("adaptive_timestep_rk4", "history_adaptive", ↵
↵ "times_adaptive", s=4),
        register_history_case("long_merger_original", "history_merger", ↵
↵ "times_merger", s=3),
        register_history_case("optimised_long_run", "history_optimised_long", ↵
↵ "times_optimised_long", s=3),
        register_history_case("differential_rotation", "history_diffrot", ↵
↵ "times_diffrot", s=3, zoom=15),
        register_history_case("interesting_showcase", "history_interesting", ↵
↵ "times_interesting", s=3, zoom=15),
        register_history_case("rk4_integrator_comparison", ↵
↵ "history_rk4_compare", "times_rk4_compare", s=3),
        register_history_case("euler_integrator_comparison", ↵
↵ "history_euler_compare", "times_euler_compare", s=3),
    ]

    if "playground_result" in globals() and isinstance(playground_result, dict):
        cases.append(
            {
                "name": f"playground_{playground_result['params']['label']}",
                "history": playground_result["history"],
            }
        )

```

```

        "times": playground_result["times"],
        "s": playground_result["params"].get("scatter_size", 3),
        "zoom": 15,
    }
)

metadata_path = DATA_DIR / "playground_current_parameters.json"
metadata_path.write_text(
    json.dumps(_json_safe(playground_result["params"]), indent=2),
    encoding="utf-8"
)
EXPORT_MANIFEST["data"].append(str(metadata_path))

if "playground_multi_results" in globals():
    for index, result in enumerate(playground_multi_results):
        cases.append(
            {
                "name": _
                ↪f"playground_case_{index}_{result['params']['label']}",
                "history": result["history"],
                "times": result["times"],
                "s": result["params"].get("scatter_size", 3),
                "zoom": 15,
            }
        )

metadata_path = DATA_DIR / "playground_multi_parameters.json"
metadata_path.write_text(
    json.dumps(
        [_json_safe(result["params"]) for result in _
        ↪playground_multi_results],
        indent=2
    ),
    encoding="utf-8"
)
EXPORT_MANIFEST["data"].append(str(metadata_path))

cases = [case for case in cases if case is not None]

if export_animations and not mpl_animation.writers.is_available("ffmpeg"):
    print("Warning: ffmpeg is not available, so animations will be saved as _
    ↪HTML files instead of MP4 videos.")
    print("Install ffmpeg and restart the notebook kernel to enable MP4 _
    ↪export.")
EXPORT_MANIFEST["notes"].append(
    "ffmpeg was not available; animation exports fell back to HTML."
)

```

```

for case in cases:
    print(f"Exporting {case['name']}")
    save_snapshot_grid(
        case["history"],
        times=case["times"],
        name=case["name"],
        s=case["s"],
        zoom=case["zoom"]
    )

    save_single_frame(
        np.asarray(case["history"])[-1],
        name=f"{case['name']}_final_state",
        title=f"{case['name']} final state",
        s=case["s"],
        zoom=case["zoom"]
    )

    if export_animations:
        save_animation_file(
            case["history"],
            times=case["times"],
            name=case["name"],
            interval=80,
            s=case["s"],
            zoom=case["zoom"],
            max_frames=max_animation_frames,
            prefer="mp4"
        )

if all(name in globals() for name in [
    "times_energy",
    "total_energy_history",
    "kinetic_energy_history",
    "potential_energy_history",
]):
    save_energy_diagnostic_plot(
        times_energy,
        total_energy_history,
        kinetic_energy=kinetic_energy_history,
        potential_energy=potential_energy_history,
        name="rk4_energy_conservation"
    )

if all(name in globals() for name in [
    "times_rk4_compare",

```

```

        "total_energy_rk4_compare",
        "kinetic_energy_rk4_compare",
        "potential_energy_rk4_compare",
    ]):
        save_energy_diagnostic_plot(
            times_rk4_compare,
            total_energy_rk4_compare,
            kinetic_energy=kinetic_energy_rk4_compare,
            potential_energy=potential_energy_rk4_compare,
            name="rk4_integrator_energy"
        )

    if all(name in globals() for name in [
        "times_euler_compare",
        "total_energy_euler_compare",
        "kinetic_energy_euler_compare",
        "potential_energy_euler_compare",
    ]):
        save_energy_diagnostic_plot(
            times_euler_compare,
            total_energy_euler_compare,
            kinetic_energy=kinetic_energy_euler_compare,
            potential_energy=potential_energy_euler_compare,
            name="euler_integrator_energy"
        )

    manifest_path = RESULTS_DIR / "export_manifest.json"
    manifest_path.write_text(
        json.dumps(_json_safe(EXPORT_MANIFEST), indent=2),
        encoding="utf-8"
    )

    print(f"Export complete. Results saved in: {RESULTS_DIR}")
    print(f"Manifest written to: {manifest_path}")

    return EXPORT_MANIFEST

# Run the export at full length. This uses every saved frame in each history.
# If full-length videos are too slow or too large, set max_animation_frames
# to a finite number such as 300, or set export_animations=False.
export_manifest = export_available_results(
    max_animation_frames=None,
    export_animations=True
)

```

Exporting fixed_timestep_rk4
 Exporting adaptive_timestep_rk4

Exporting long_merger_original
Exporting optimised_long_run
Exporting differential_rotation
Exporting interesting_showcase
Exporting rk4_integrator_comparison
Exporting euler_integrator_comparison
Exporting playground_Best Merger?
Exporting playground_case_0_head-on low values
Exporting playground_case_1_grazing merger
Exporting playground_case_2_Fly Through
Export complete. Results saved in: /Users/williamwall/Documents/Amsterdam/UvA/Period5/Computational/Comp_Project/results
Manifest written to: /Users/williamwall/Documents/Amsterdam/UvA/Period5/Computational/Comp_Project/results/export_manifest.json